

School of Computing Science and Engineering
VIT Bhopal University

Computer Vision
CSE3010 (Lab File)
Fall Semester July 2026

Index					
S. No.	Description of Experiment	Date	Page no.	Remarks	Signature of Faculty
1	Implement image preprocessing and Edge detection		2		
2	Implement camera calibration methods		7		
3	Implement Projection		12		
4	Determine depth map from Stereo pair		16		
5	Construct 3D model from Stereo pair		20		
6	Implement Segmentation methods		24		
7	Construct 3D model from defocus image		29		
8	Construct 3D model from Images		33		
9	Implement optical flow method		37		
10	Implement object detection and tracking from video		42		
11	Face detection and Recognition		46		
12	Object detection from dynamic Background for Surveillance		50		
13	Content based video retrieval		55		
14	Construct 3D model from single image		60		

Submitted by

Student Name:.....

Registration number:.....

Submitted to

Faculty Name: Dr. Siddharth Singh Chouhan

Signature:

Experiment No. 1

Implement image preprocessing and Edge detection

Experiment No.	1
Title	Implement image preprocessing and Edge detection
Date of experiment	
Tools / software used	Python 3.11, OpenCV 5.0, NumPy, Matplotlib
Input data	A grape leaf with black rot lesions from the PlantVillage dataset (256 x 256, resized to 512 x 512) and the standard lena.jpg test image from the OpenCV samples.
Program file	exp01_preprocessing_edges.py

1. Aim

To implement the basic preprocessing operations (colour conversion, resizing, histogram equalisation, smoothing, sharpening and morphology) and to detect edges with the Sobel, Prewitt, Roberts, Laplacian, LoG and Canny operators in OpenCV.

2. Objectives

- Improve contrast with histogram equalisation and CLAHE.
- Remove Gaussian and salt-and-pepper noise with Gaussian, median and bilateral filters and measure the PSNR.
- Detect edges with Sobel, Prewitt, Roberts, Laplacian, LoG and Canny.

3. Theory

Preprocessing converts the BGR image to grayscale, resizes it and improves the contrast. Histogram equalisation maps each intensity through the cumulative distribution, so the output histogram is nearly flat. CLAHE does the same per tile with a clip limit, which keeps noise from being amplified as much. A Gaussian filter suppresses Gaussian noise but blurs edges. The median filter removes impulse noise, and the bilateral filter smooths flat regions but leaves edges alone. Filter quality is measured with the peak signal-to-noise ratio (PSNR).

$$\text{PSNR} = 10 \log_{10} \frac{255^2}{\text{MSE}}, \quad \text{MSE} = \frac{1}{MN} \sum_{x,y} (f(x,y) - g(x,y))^2$$

Edges are found from derivatives. Sobel, Prewitt and Roberts estimate the gradient components G_x and G_y with small kernels. The Laplacian is a second derivative whose zero crossings mark edges; because it amplifies noise it is combined with Gaussian smoothing (LoG). Canny smooths the image, computes the gradient, keeps only local maxima along the gradient direction and then applies two hysteresis thresholds. The result is a set of thin, connected edges.

$$G = \sqrt{G_x^2 + G_y^2}, \quad \theta = \text{atan2}(G_y, G_x)$$

4. Algorithm / Procedure

1. Read the leaf image, convert BGR to RGB and grayscale, resize to 512×512 .
2. Apply histogram equalisation and CLAHE (clip 2, 8×8 tiles); plot histograms and contrast statistics.
3. Apply Gaussian blur, unsharp-mask sharpening and morphological opening and closing.
4. Add Gaussian ($\sigma = 20$) and salt-and-pepper noise, denoise with Gaussian, median and bilateral filters and compute the PSNR.
5. Compute Sobel, Prewitt, Roberts, Laplacian and LoG responses and their edge-pixel percentages.
6. Run Canny with thresholds (30, 90), (60, 160) and (120, 260) on the leaf and lena and save the figures.

5. Output

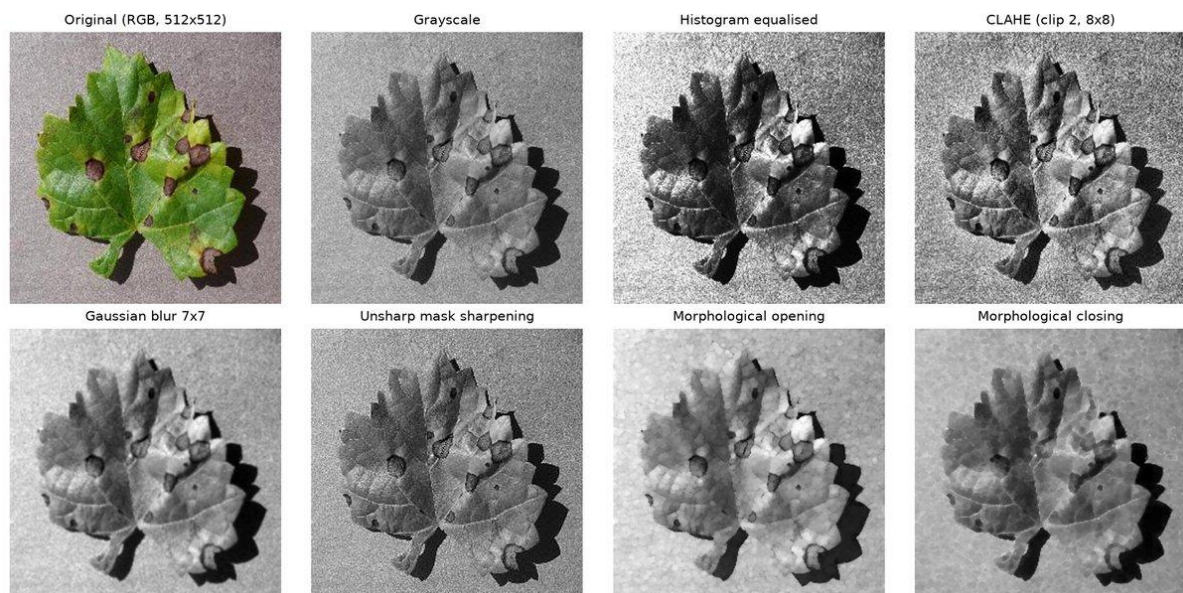


Figure 1.1: Preprocessing of the grape leaf: original, grayscale, equalised, CLAHE, Gaussian blur, sharpening, opening and closing.

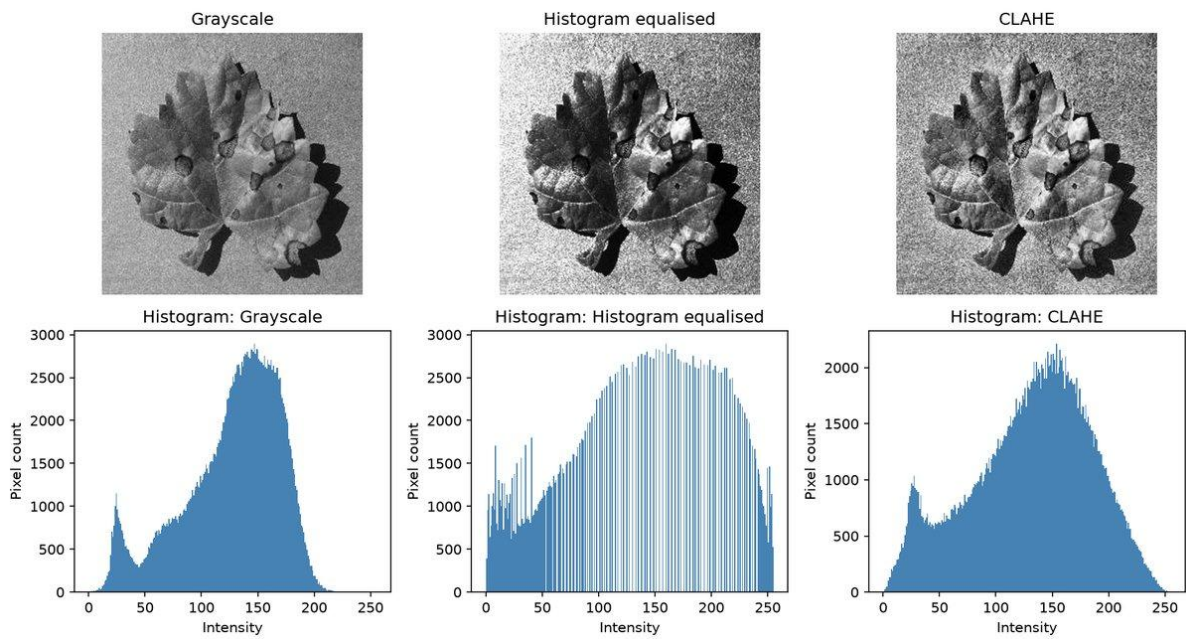


Figure 1.2: Grayscale, equalised and CLAHE images with their histograms.

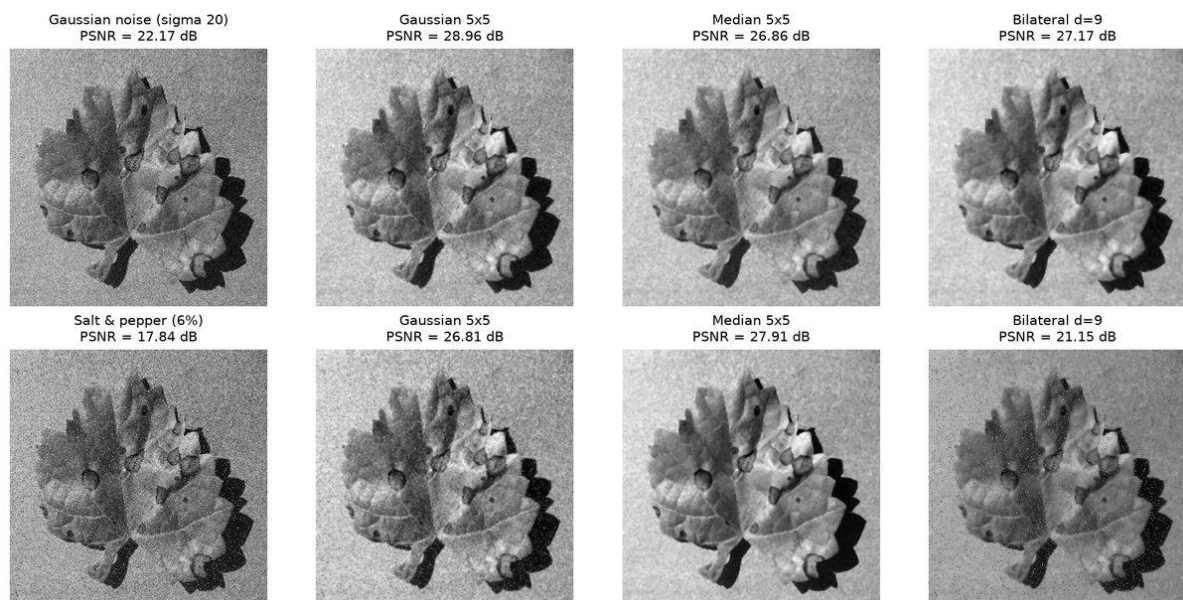


Figure 1.3: Gaussian (top) and salt-and-pepper (bottom) noise filtered with Gaussian, median and bilateral filters. PSNR in the titles.

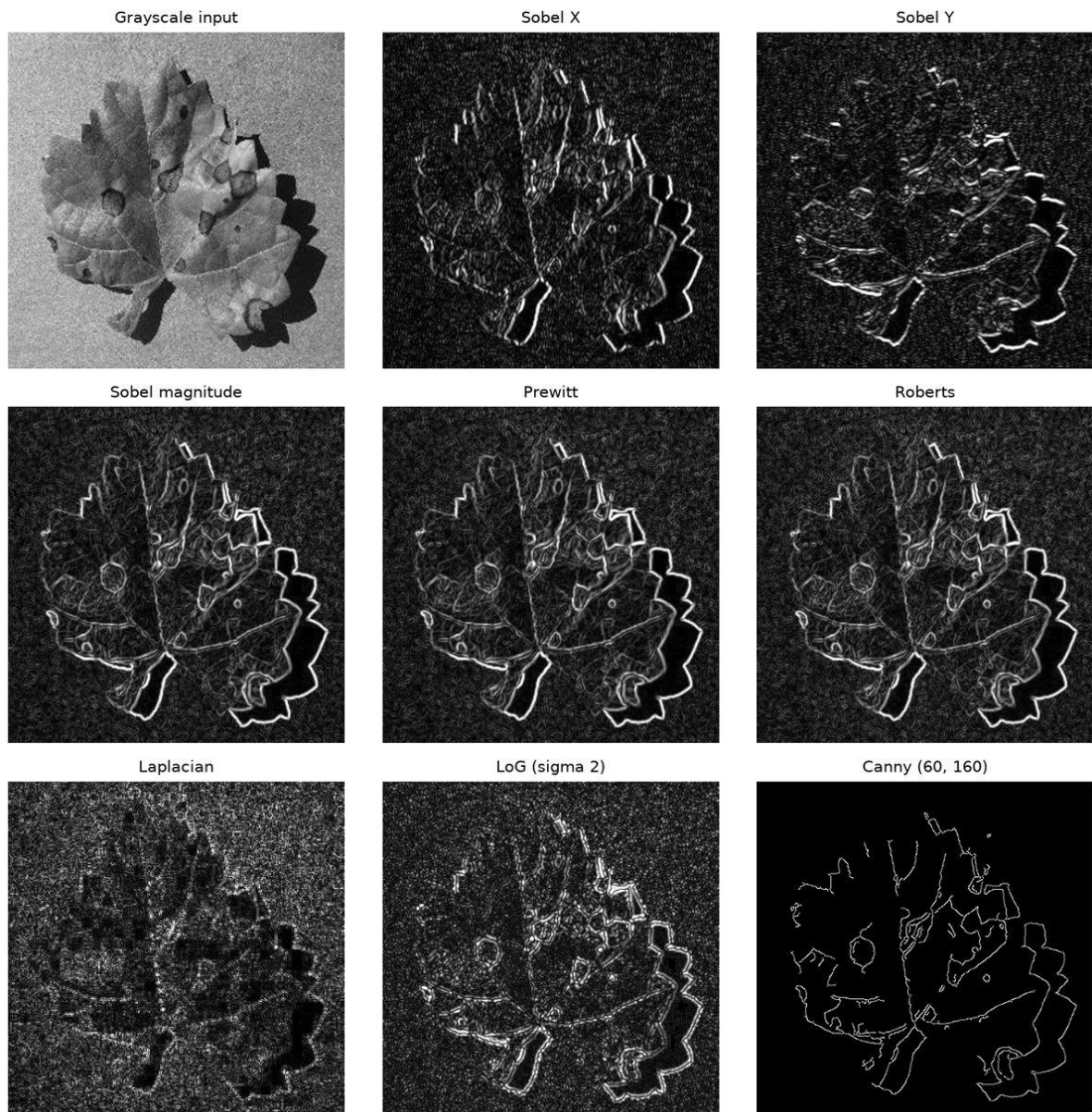


Figure 1.4: Edge responses of Sobel X, Sobel Y, Sobel magnitude, Prewitt, Roberts, Laplacian, LoG and Canny.

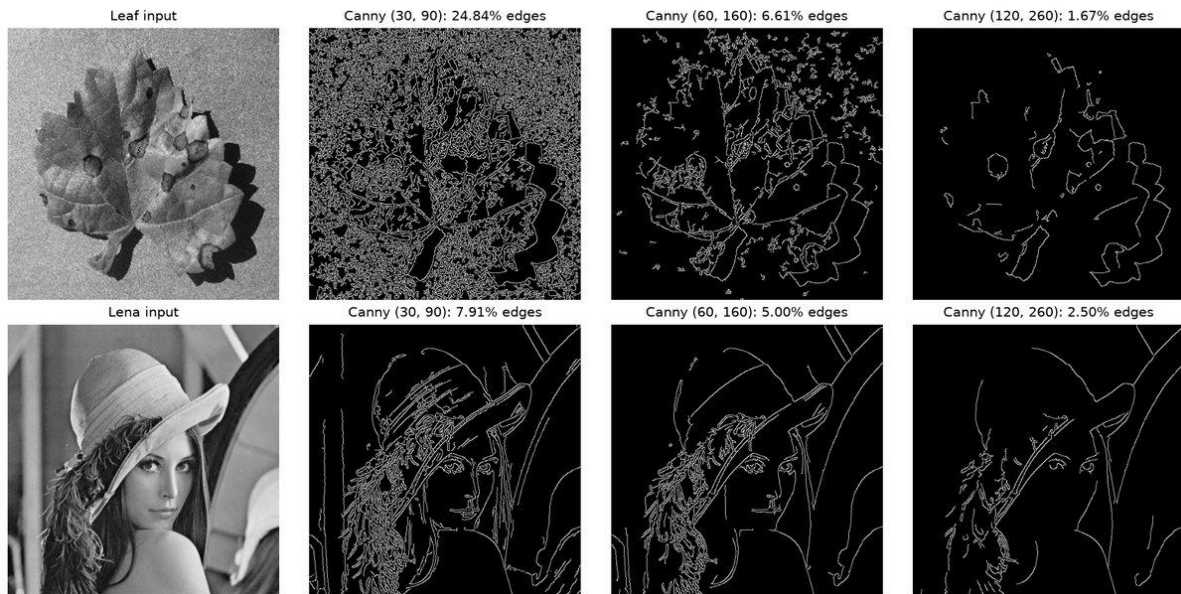


Figure 1.5: Effect of the Canny hysteresis thresholds on the leaf and lena images.

Console output

```
Original leaf image shape: (256, 256, 3) dtype: uint8
Resized to: (512, 512, 3)  grayscale mean = 127.2, std = 42.8
gray      mean = 127.2  std = 42.8  contrast (p95-p5) = 149
equalised mean = 128.4  std = 73.8  contrast (p95-p5) = 230
CLAHE     mean = 130.8  std = 53.5  contrast (p95-p5) = 182
Gaussian noise (sigma 20) -> Gaussian 5x5  PSNR = 28.96 dB
Gaussian noise (sigma 20) -> Median 5x5    PSNR = 26.86 dB
Gaussian noise (sigma 20) -> Bilateral d=9  PSNR = 27.17 dB
Salt & pepper (6%)       -> Gaussian 5x5  PSNR = 26.81 dB
Salt & pepper (6%)       -> Median 5x5    PSNR = 27.91 dB
Salt & pepper (6%)       -> Bilateral d=9  PSNR = 21.15 dB

Edge pixel percentage (a pixel is an edge if |response| > 25 % of the maximum response):
Sobel X      3.77 %
Sobel Y      2.68 %
Sobel magnitude  6.44 %
Prewitt      6.43 %
Roberts      5.97 %
... (output truncated) ...
```

6. Results and Observations

Histogram equalisation raised the grayscale standard deviation from 42.8 to 73.8 and the 5-95 percentile range from 149 to 230. CLAHE gave 53.5 and 182, with less noise amplification.

For Gaussian noise (input 22.17 dB) the Gaussian filter was best at 28.96 dB. For salt-and-pepper noise (input 17.84 dB) the median filter was best at 27.91 dB; the bilateral filter reached only 21.15 dB.

Edge pixels at 25 percent of the maximum response: Sobel 6.44, Prewitt 6.43, Roberts 5.97, LoG 4.00 and Laplacian 1.74 percent. Canny gave 24.84, 6.61 and 1.67 percent for the thresholds (30, 90), (60, 160) and (120, 260).

7. Conclusion

Equalisation and CLAHE improve contrast, the Gaussian filter suits Gaussian noise and the median filter suits impulse noise. Canny gives the thinnest edges, and its hysteresis thresholds set how much detail survives against the noise.

Experiment No. 2

Implement camera calibration methods

Experiment No.	2
Title	Implement camera calibration methods
Date of experiment	
Tools / software used	Python 3.11, OpenCV 5.0 (calib3d module), NumPy, Matplotlib
Input data	13 grayscale JPEG images (640 x 480) of a 9 x 6 inner-corner chessboard, left01.jpg to left14.jpg from the OpenCV samples/data repository; the square size is taken as 25 mm.
Program file	exp02_camera_calibration.py

1. Aim

To estimate the intrinsic parameters, lens distortion coefficients and extrinsic poses of a camera from several views of a planar chessboard with Zhang's method, and to check the result with the reprojection error and an undistorted image.

2. Objectives

- Detect chessboard corners and refine them to sub-pixel accuracy.
- Estimate K and the distortion coefficients with `cv2.calibrateCamera` and check the reprojection error.
- Undistort an image and save the calibration for later experiments.

3. Theory

A camera maps a world point $\mathbf{X}$ to a pixel $\mathbf{x}$ by $\mathbf{x} \sim K[R|t]\mathbf{X}$. K holds the focal lengths f_x, f_y , the principal point (c_x, c_y) and the skew s . $[R|t]$ is the pose of one view.

$$\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} \sim K[R|t] \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}, \quad K = \begin{bmatrix} f_x & s & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

Real lenses add radial (k_1, k_2, k_3) and tangential (p_1, p_2) distortion, applied to normalised coordinates. A negative k_1 means barrel distortion.

$$x_d = x(1 + k_1 r^2 + k_2 r^4 + k_3 r^6) + 2p_1 xy + p_2(r^2 + 2x^2), \quad r^2 = x^2 + y^2$$

Zhang's method needs a planar chessboard photographed in at least three orientations. Each view gives a homography $H = K[r_1 r_2 t]$, and the orthonormality of r_1 and r_2 fixes K in closed form. A Levenberg-Marquardt refinement then minimises the RMS reprojection error of the corners. `cv2.undistort` resamples the image with the recovered coefficients.

4. Algorithm / Procedure

1. Define the 54 inner corners of the board in its own frame (X, Y multiples of 25 mm, $Z = 0$).

2. Detect the corners in each image with `cv2.findChessboardCorners` and refine them with `cv2.cornerSubPix`.
3. Call `cv2.calibrateCamera` on all views to obtain the RMS error, K , the distortion vector and the poses.
4. Reproject the corners with `cv2.projectPoints`, compute the error of each image and derive the field of view.
5. Convert the rotation vector of view 1 with `cv2.Rodrigues` and compute the camera centre $C = -R^T t$.
6. Undistort `left01.jpg` with `cv2.undistort` and again with `initUndistortRectifyMap` and `remap`, and measure the pixel displacement.
7. Save the calibration to `outputs/exp02/calibration.npz` and plot the corners, errors, undistortion and camera poses.

5. Output

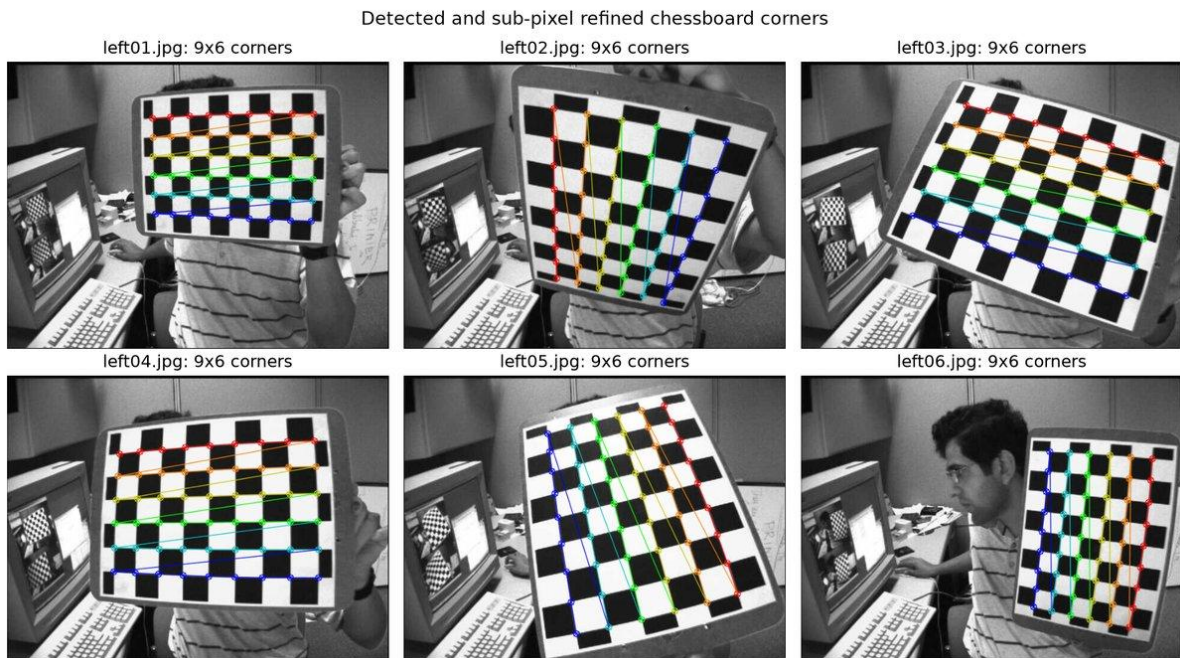


Figure 2.1: Detected and sub-pixel refined chessboard corners in six of the 13 calibration views.

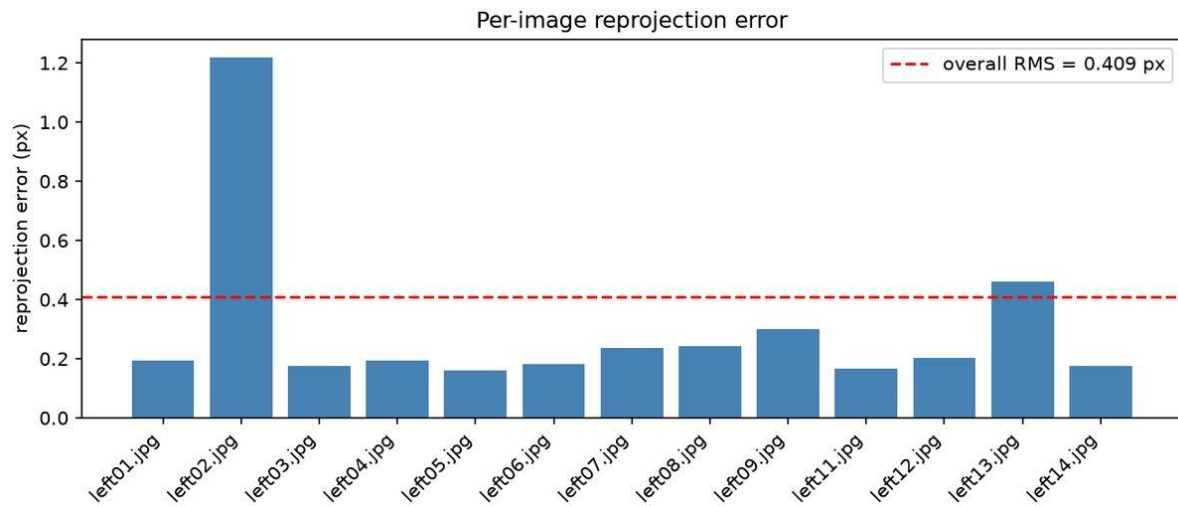


Figure 2.2: RMS reprojection error of each image; the dashed line is the overall RMS of 0.409 px.

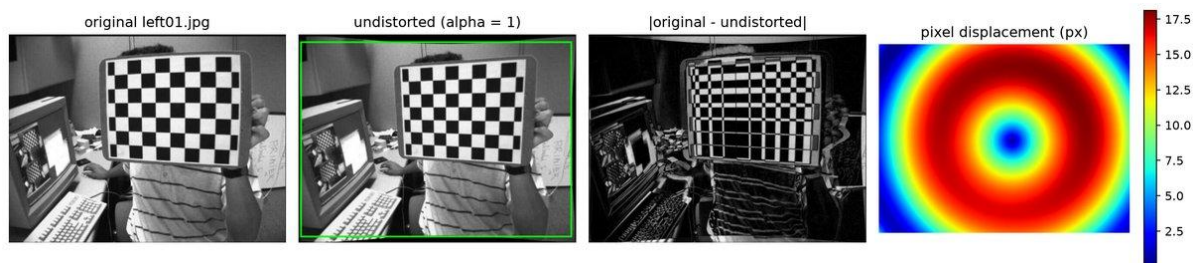


Figure 2.3: Original image, undistorted image ($\alpha = 1$), their difference and the displacement map of the distortion (up to 18.1 px).

Estimated camera poses (extrinsics) of the 13 views relative to the board

— chessboard (Z = 0)

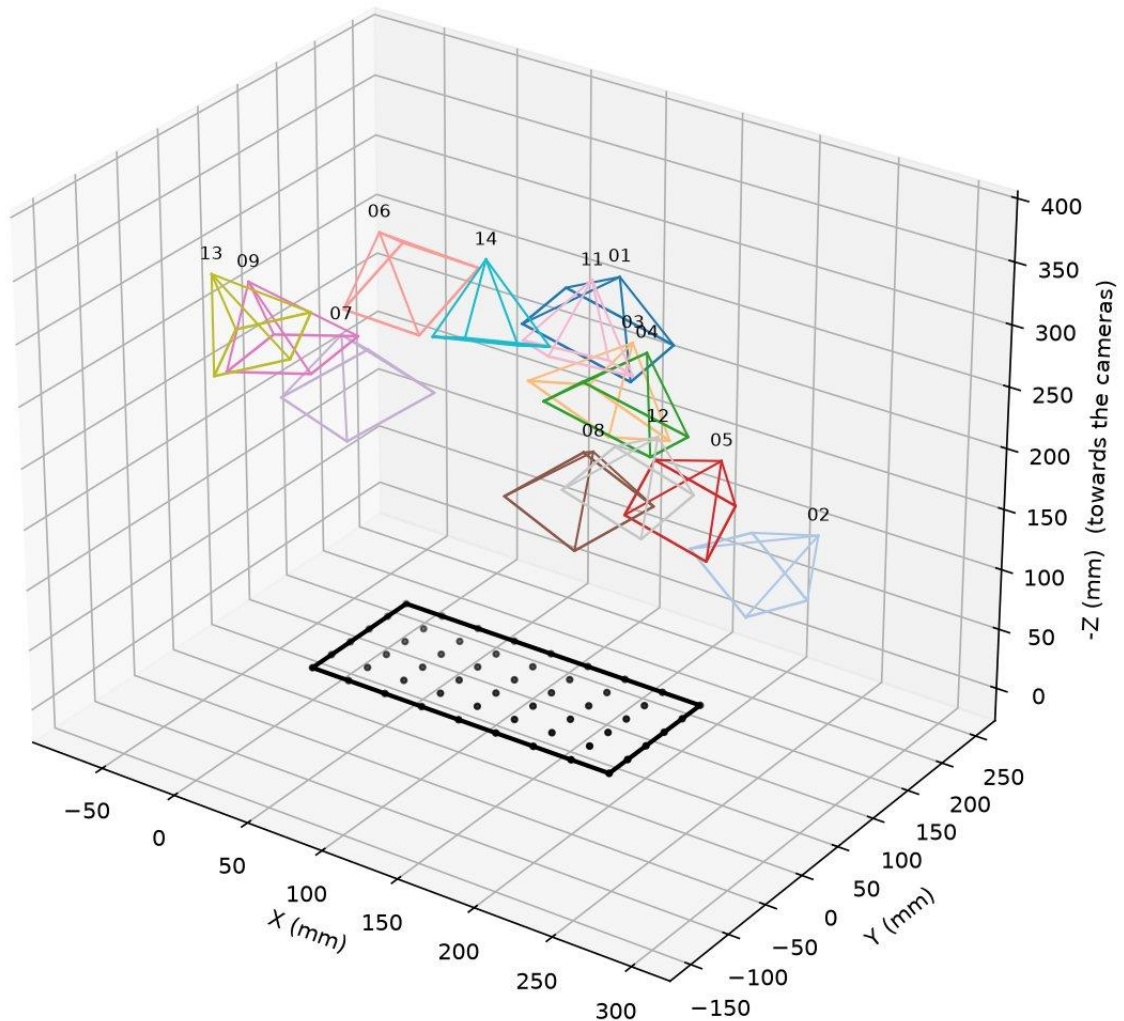


Figure 2.4: Estimated camera poses of the 13 views drawn as frustums in the chessboard frame.

Console output

```

Images used: 13 of 13, image size 640x480, pattern 9x6, square 25.0 mm
RMS reprojection error: 0.4087 px
Camera matrix K:
[[536.0735  0.      342.3705]
 [ 0.      536.0164 235.5369]
 [ 0.      0.      1.      ]]
Distortion coefficients [k1 k2 p1 p2 k3]: [-0.2651 -0.0467 0.0018 -0.0003 0.2523]
fx = 536.07 px, fy = 536.02 px, cx = 342.37 px, cy = 235.54 px
Field of view: 61.60 deg (horizontal) x 48.24 deg (vertical); focal length = 4.02 mm for a 4.8 x 3.6
mm sensor; aspect ratio fy/fx = 0.9999
left01.jpg: reprojection error = 0.1934 px
left02.jpg: reprojection error = 1.2198 px
left03.jpg: reprojection error = 0.1754 px
left04.jpg: reprojection error = 0.1940 px
left05.jpg: reprojection error = 0.1594 px
left06.jpg: reprojection error = 0.1826 px
left07.jpg: reprojection error = 0.2375 px
left08.jpg: reprojection error = 0.2434 px
left09.jpg: reprojection error = 0.3006 px
... (output truncated) ...

```

6. Results and Observations

All 13 images were accepted. The RMS reprojection error is 0.4087 px, with $f_x = 536.07$ px, $f_y = 536.02$ px, principal point (342.37, 235.54) px, $k_1 = -0.2651$, $k_2 = -0.0467$, $p_1 = 0.0018$, $p_2 = -0.0003$ and $k_3 = 0.2523$. The negative k_1 means barrel distortion.

The field of view is $61.6^\circ \times 48.2^\circ$, a focal length of 4.02 mm for a 4.8×3.6 mm sensor. Per-image errors lie between 0.16 and 0.30 px, except left13.jpg at 0.46 px and left02.jpg at 1.22 px. left01.jpg is 421.2 mm from the board, with $t = (-75.28, -108.94, 399.82)$ mm.

Undistortion moves pixels by 12.57 px on average and by 18.10 px at the corners. cv2.undistort and the remap route differ by at most 4 grey levels.

7. Conclusion

Zhang's method recovered K , the distortion coefficients and all 13 poses with an RMS error of 0.41 px. The camera positions are plausible, the board edges are straight after undistortion, and the saved calibration is reused in experiments 3 and 8.

Experiment No. 3

Implement Projection

Experiment No.	3
Title	Implement Projection
Date of experiment	
Tools / software used	Python 3.11, OpenCV 5.0, NumPy, Matplotlib; intrinsic parameters loaded from outputs/exp02/calibration.npz
Input data	Synthetic 3D house wireframe (10 vertices, 17 edges); chessboard images left01, left03, left05 and left08 (640 x 480) and the sudoku.png photograph (558 x 563) from the OpenCV samples/data repository.
Program file	exp03_projection.py

1. Aim

To implement the perspective projection of 3D points onto the image plane with the pinhole model $\mathbf{x} = K [R | t] \mathbf{X}$, to compare it with orthographic projection and with `cv2.projectPoints`, and to use projection for augmented-reality overlays on calibrated images and for planar homographies.

2. Objectives

- Project a 3D wireframe with $\mathbf{x} = K[R|t]\mathbf{X}$ in NumPy and verify the result against `cv2.projectPoints`.
- Compare perspective with orthographic projection and estimate a pose with `cv2.solvePnP` for augmented reality.
- Rectify a planar scene with a homography and synthesise a rotated view.

3. Theory

In the pinhole model a point (X_c, Y_c, Z_c) is divided by its depth. That division produces foreshortening and vanishing points. In homogeneous coordinates the projection is linear, $\mathbf{x} \sim K[R|t]\mathbf{X}$, and the camera centre is $C = -R^T t$.

$$u = f_x \frac{X_c}{Z_c} + c_x, \quad v = f_y \frac{Y_c}{Z_c} + c_y$$

Orthographic projection drops the depth, $u = sX_c + c_x$, $v = sY_c + c_y$, so image size does not change with distance. The perspective-n-point problem (`cv2.solvePnP`) is the inverse: given known 3D points and their pixels it recovers R and t . With that pose, a virtual object defined in the board frame can be projected into the photograph.

When the scene is a plane the projection reduces to a 3×3 homography H with eight degrees of freedom, fixed by four correspondences. A pure camera rotation is also a homography.

$$\begin{bmatrix} u' \\ v' \\ 1 \end{bmatrix} \sim \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & 1 \end{bmatrix} \begin{bmatrix} u \\ v \\ 1 \end{bmatrix}, \quad H_{\text{rot}} = K R K^{-1}$$

4. Algorithm / Procedure

1. Load K from the calibration experiment and define the house vertices in metres.
2. Build $R = R_x(115^\circ)R_z(-35^\circ)$ and $t = (0,0,d)^T - RX_o$ for $d = 4, 8$ and 20 m.
3. Project the vertices in NumPy, scale f by $d/6$ so the house keeps its image size, and compare with `cv2.projectPoints`.
4. Project the same pose orthographically and compare the front and back edge lengths of the house.
5. Solve the pose of four chessboard images with `cv2.solvePnP` and draw the board axes and a 75 mm cube.
6. Compute the homography from the sudoku corners to a 450×450 square and rectify the image with `cv2.warpPerspective`.
7. Map the 9×9 grid back with H^{-1} and synthesise a 25 degree pan, 10 degree tilt view with KRK^{-1} .

5. Output

Pinhole projection $x = K[R|t]X$ (lines: NumPy, dots: `cv2.projectPoints`). Zooming keeps the size constant while foreshortening decreases with distance

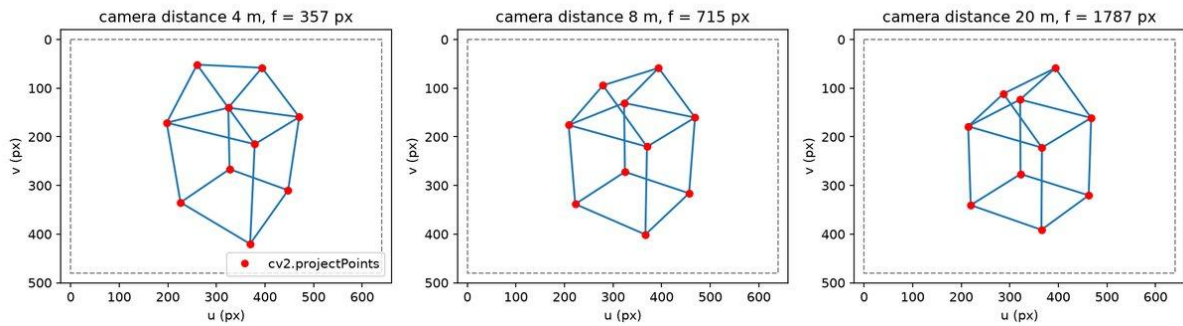


Figure 3.1: Perspective projection of the house (NumPy lines, `cv2.projectPoints` dots) at camera distances of 4, 8 and 20 m.

Same camera pose: perspective vs orthographic projection of the house

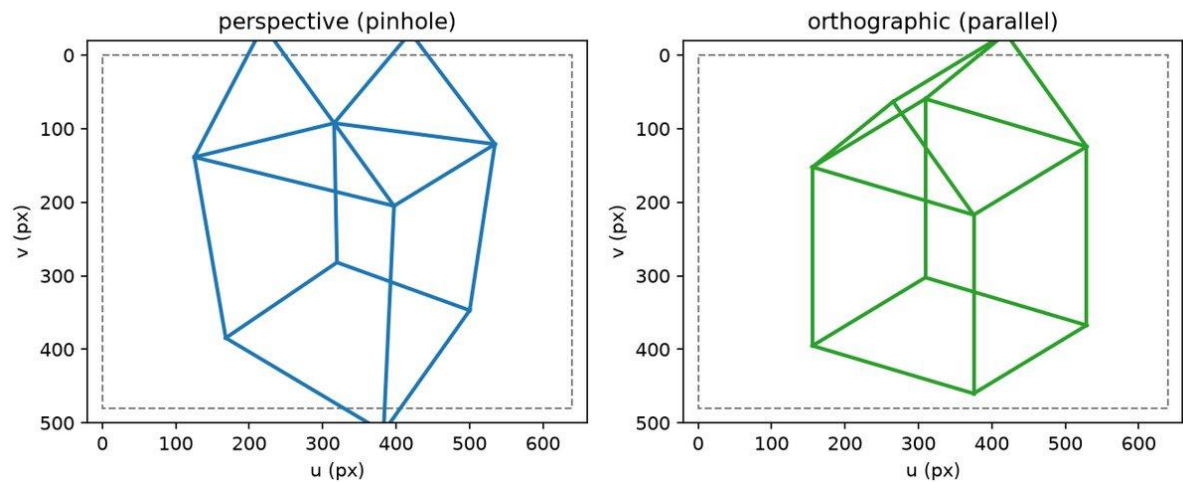
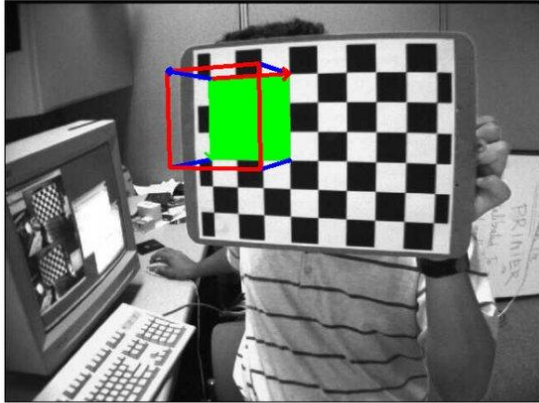


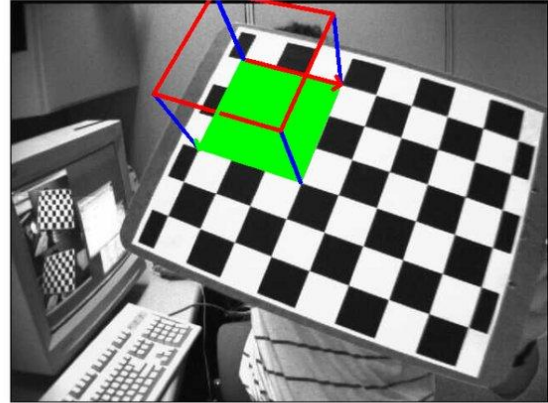
Figure 3.2: The same pose under perspective (left) and orthographic (right) projection.

Augmented reality: 3D axes and a 75 mm cube projected with the solvePnP pose

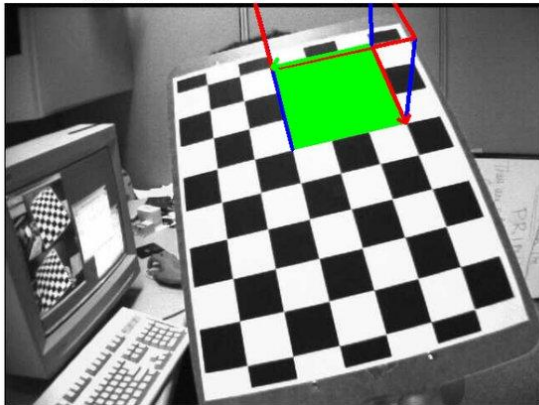
left01.jpg |t| = 421 mm, rvec = [0.17 0.28 0.01]



left03.jpg |t| = 336 mm, rvec = [-0.28 0.19 0.35]



left05.jpg |t| = 343 mm, rvec = [-0.29 0.43 1.31]



left08.jpg |t| = 338 mm, rvec = [-0.09 0.48 1.75]

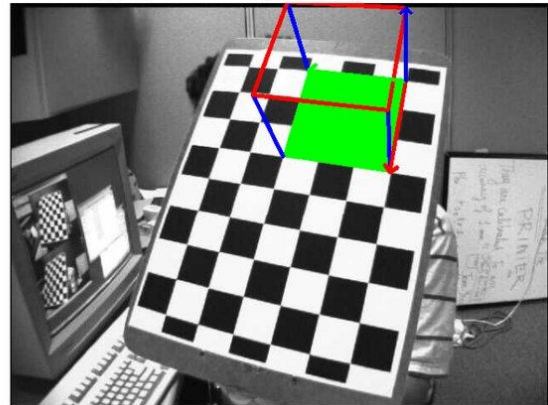


Figure 3.3: Board axes and a 75 mm cube projected onto four views with the pose from cv2.solvePnP.

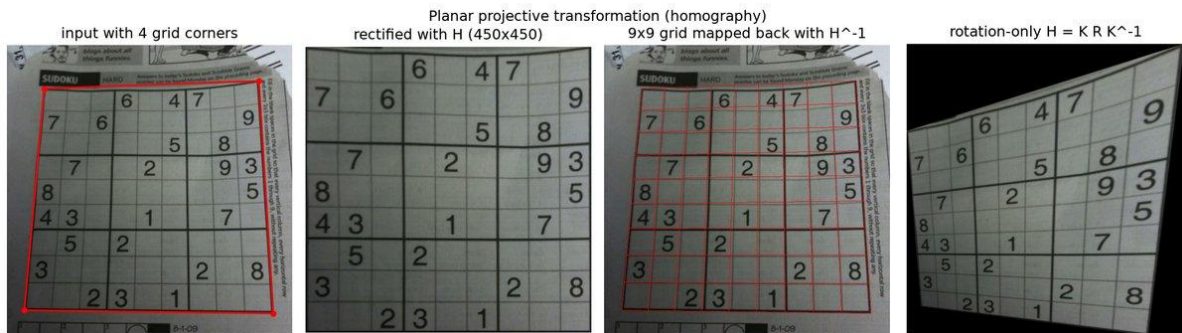


Figure 3.4: Sudoku rectified by a homography, the grid mapped back with H^{-1} , and the view from a rotation-only homography.

Console output

```
Intrinsic matrix K:
[[536.0735  0.  342.3705]
 [ 0.  536.0164 235.5369]
 [ 0.  0.  1.  ]]
Rotation from Euler angles (rx=115, rz=-35):
[[ 0.8192  0.5736  0.  ]
 [ 0.2424 -0.3462 -0.9063]
 [-0.5198  0.7424 -0.4226]]
camera centre in world coordinates for d = 4.0 m: [ 2.08 -2.97  2.89]
distance 4.0 m, f = 357.4 px: max |manual - cv2.projectPoints| = 8.53e-14 px
camera centre in world coordinates for d = 8.0 m: [ 4.16 -5.94  4.58]
distance 8.0 m, f = 714.8 px: max |manual - cv2.projectPoints| = 8.53e-14 px
```

```

camera centre in world coordinates for d = 20.0 m: [ 10.4 -14.85  9.65]
distance 20.0 m, f = 1786.9 px: max |manual - cv2.projectPoints| = 8.53e-14 px
Perspective: front edge 250.0 px, back edge 192.1 px (ratio 1.30); orthographic: 229.0 px and 229.0
px
solvePnP left01: rvec = [0.1685 0.2758 0.0135], tvec (mm) = [ -75.2797 -108.9392  399.8218]
solvePnP left03: rvec = [-0.277  0.1869  0.3548], tvec (mm) = [ -39.8955 -100.4004  318.2425]
solvePnP left05: rvec = [-0.2919  0.4283  1.3127], tvec (mm) = [  58.4416 -115.3019  317.269 ]
... (output truncated) ...

```

6. Results and Observations

NumPy projection and cv2.projectPoints agree to 8.5×10^{-14} px. At $d = 4$ m the front edge of the house measures 250.0 px and the back edge 192.1 px, a ratio of 1.30. Under orthographic projection both edges are 229.0 px.

cv2.solvePnP returned $t = (-75.28, -108.94, 399.82)$ mm for left01.jpg, the same as the calibration extrinsics. The four board distances are 421, 336, 343 and 338 mm, and the cube sits on the board in every view.

$$H = \begin{bmatrix} 1.1753 & 0.1061 & -94.7089 \\ 0.0451 & 1.2595 & -109.0918 \\ 0.0001 & 0.0004 & 1.0000 \end{bmatrix}$$

7. Conclusion

The hand-written projection matched OpenCV to machine precision, and foreshortening decreased with distance until the drawing approached the orthographic limit. A solvePnP pose placed a virtual cube on real photographs, and the homography rectified the sudoku grid and synthesised a rotated view.

Experiment No. 4

Determine depth map from Stereo pair

Experiment No.	4
Title	Determine depth map from Stereo pair
Date of experiment	
Tools / software used	Python 3.11, OpenCV 5.0 (StereoBM, StereoSGBM), NumPy, Matplotlib
Input data	Middlebury 2006 'Aloe' rectified stereo pair (data/aloeL.jpg and data/aloeR.jpg, 1282 x 1110 px, taken from the OpenCV sample data) processed at half resolution (641 x 555 px).
Program file	exp04_stereo_depth.py

1. Aim

To compute a dense disparity map from a rectified stereo pair with block matching and semi-global block matching, and to convert it into a depth map from the geometry of the rig.

2. Objectives

- To understand the relation $Z = fB/d$ between depth and disparity for a rectified stereo pair.
- To compute disparity maps with StereoBM and StereoSGBM and to see what the block size and disparity range change.
- To convert the disparity map into metric depth.

3. Theory

A rectified stereo rig has two cameras a baseline B apart with parallel optical axes. Corresponding points then lie on the same image row, and their column difference $d = x_L - x_R$ is the disparity. Similar triangles give the depth $Z = fB/d$, where f is the focal length in pixels. One pixel of disparity error is about $Z^2/(fB)$ of depth error, so resolution worsens with the square of the distance.

$$Z = \frac{fB}{d}, \quad d = x_L - x_R, \quad \Delta Z \approx \frac{Z^2}{fB} \Delta d$$

Finding the matching pixel along the row is the correspondence problem. StereoBM compares small windows with the sum of absolute differences and keeps the cheapest disparity: fast, but noisy where the image has no texture. StereoSGBM adds a smoothness penalty, P_1 for a one-pixel change and P_2 for larger jumps, and minimises the energy by dynamic programming along several image directions, which gives denser, cleaner maps. Failed matches are marked invalid, and a median filter removes isolated errors.

$$E(D) = \sum_p C(p, D_p) + \sum_{q \in N_p} P_1 [|D_p - D_q| = 1] + \sum_{q \in N_p} P_2 [|D_p - D_q| > 1]$$

4. Algorithm / Procedure

1. Read the left and right images, resize them to half resolution and convert to grayscale.

2. Compute the StereoBM disparity (numDisparities 112, blockSize 15) and divide the fixed-point result by 16.
3. Compute the StereoSGBM disparity (blockSize 5, $P_1 = 600$, $P_2 = 2400$, uniqueness ratio 10, 3-way mode).
4. Mark disparities ≤ 0 as invalid and apply a 5×5 median filter.
5. Repeat SGBM for blockSize 3, 7, 11 and for numDisparities 48, 80, 112.
6. Convert disparity to depth with $Z = fB/(d + d_{\min})$, $f = 1870$ px, $B = 160$ mm, $d_{\min} = 135$ px.
7. Plot the pair, both disparity maps, the parameter sweep and the depth map. Save the disparity for experiment 5.

5. Output



Figure 4.1: Rectified Aloe stereo pair at half resolution. Corresponding points lie on the same row.

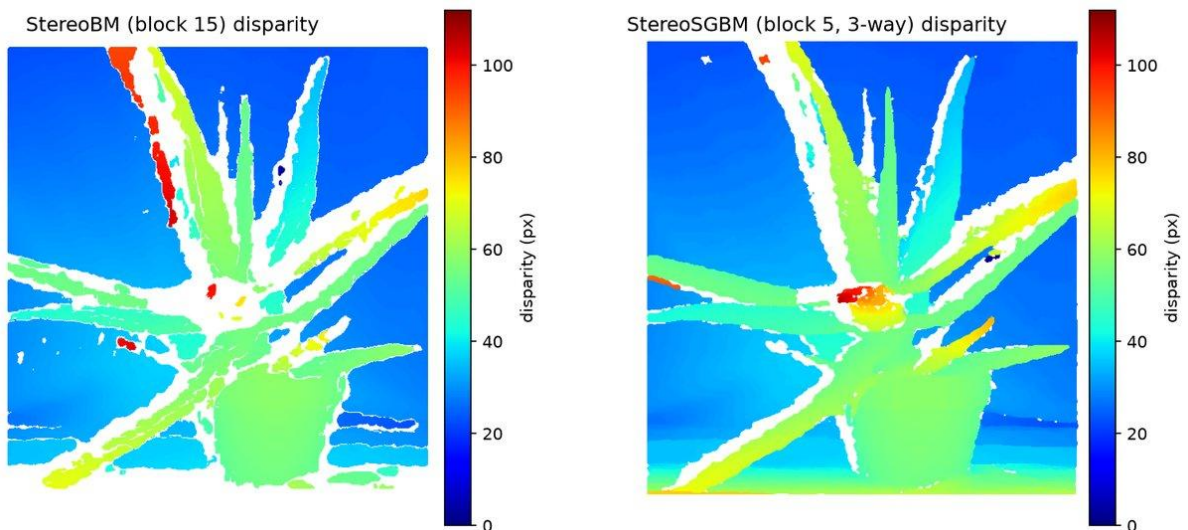


Figure 4.2: StereoBM (left) and StereoSGBM (right) disparity maps. White pixels are invalid matches.

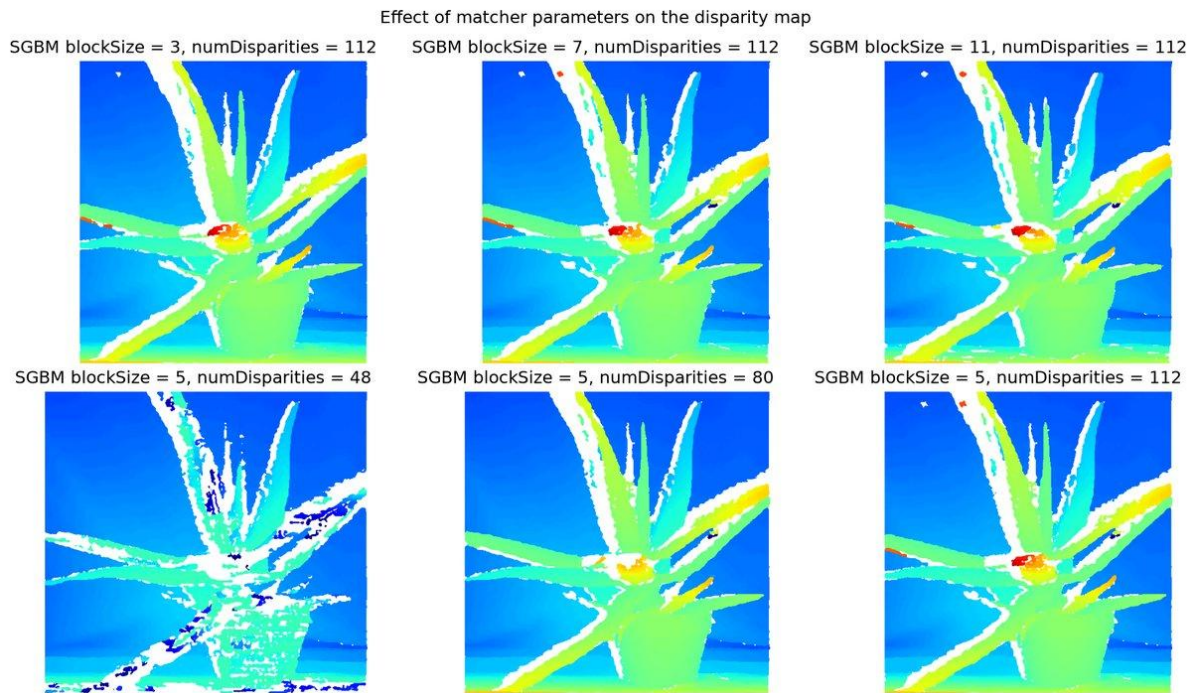


Figure 4.3: Effect of blockSize (top) and numDisparities (bottom) on the SGBM disparity map.

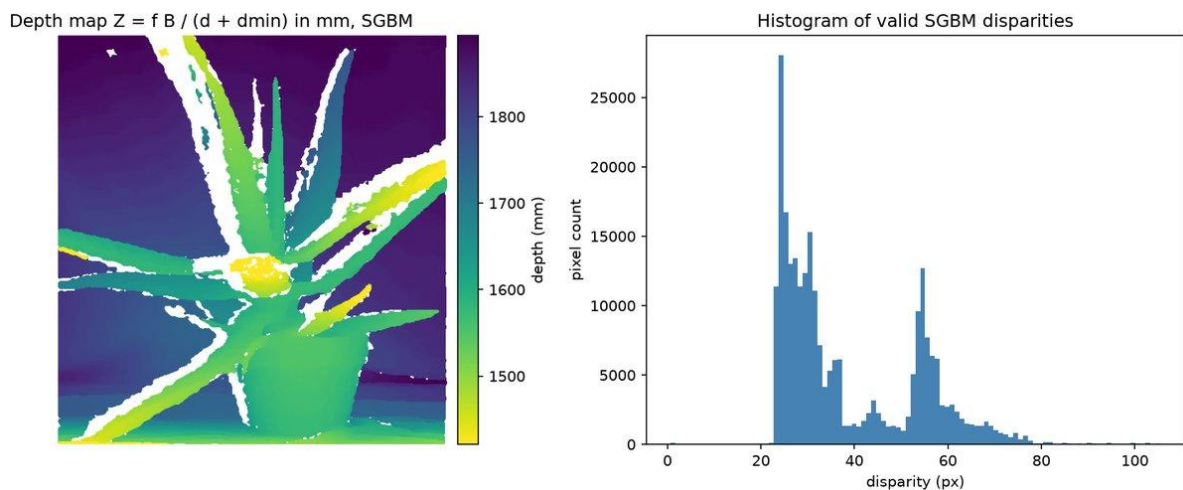


Figure 4.4: Depth map from the SGBM disparity with $Z = fB / (d + d_{\min})$, and the histogram of valid disparities.

Console output

```
Stereo pair loaded and scaled by 0.5: 641 x 555 pixels
StereoBM : time 15.5 ms, valid pixels 61.8 %, disparity min 3.2, max 103.7, mean 37.3 px
StereoSGBM: time 38.4 ms, valid pixels 73.2 %, disparity min 0.7, max 105.4, mean 38.7 px
Depth from SGBM (f = 1870 px, B = 160 mm, dmin = 135 px): near (1st percentile) 1421 mm, far (99th
percentile) 1894 mm, median 1792 mm
Figures written to E:\Plant-Disease-Detection-and-Analysis-System\LAB\outputs\exp04
```

6. Results and Observations

StereoBM returned valid disparities for 61.8 % of the pixels (3.2 to 103.7 px) in about 15 ms. StereoSGBM returned 73.2 % valid pixels (0.7 to 105.4 px) in about 38 ms at 641×555 .

SGBM is smoother on the cloth and the pot. BM leaves holes and wrong patches in the low-texture areas. Small blocks keep the thin leaf tips, and numDisparities must cover the largest true disparity of about 105 px: with 48 the near leaves are lost.

With $f = 1870$ px, $B = 160$ mm and $d_{\min} = 135$ px the depth runs from 1421 mm to 1894 mm (1st to 99th percentile), with a median of 1792 mm.

7. Conclusion

Dense correspondence between the two images of a rectified pair gives a disparity map, and $Z = fB/d$ turns it into depth. Block matching is fast but fails without texture; semi-global matching costs about twice the time and gives a denser, cleaner map that converts directly to metric depth.

Experiment No. 5

Construct 3D model from Stereo pair

Experiment No.	5
Title	Construct 3D model from Stereo pair
Date of experiment	
Tools / software used	Python 3.11, OpenCV 5.0 (reprojectImageTo3D, StereoSGBM), NumPy, Matplotlib (mplot3d, Poly3DCollection)
Input data	SGBM disparity map and half-resolution left image of the Middlebury Aloe pair produced in experiment 4 (recomputed from data/aloeL.jpg and data/aloeR.jpg when the files are missing).
Program file	exp05_3d_from_stereo.py

1. Aim

To reconstruct a three-dimensional model of a scene, as a coloured point cloud and a triangle mesh, from the disparity map of a rectified stereo pair using the reprojection matrix Q .

2. Objectives

- To derive the reprojection matrix Q that maps pixel coordinates and disparity to 3D points.
- To build a coloured point cloud and a triangle mesh from the disparity map and export a PLY file.
- To view and interpret the reconstruction from several viewpoints.

3. Theory

For a rectified rig with focal length f , principal point (c_x, c_y) and baseline B , a pixel (x, y) with disparity d maps to $X = (x - c_x)B/d$, $Y = (y - c_y)B/d$, $Z = fB/d$. In homogeneous coordinates these relations form the 4×4 reprojection matrix Q , which `cv2.reprojectImageTo3D` applies to every pixel. With the left-image colour attached, the result is a point cloud, stored as a PLY file.

$$\begin{bmatrix} X \\ Y \\ Z \\ W \end{bmatrix} = Q \begin{bmatrix} x \\ y \\ d \\ 1 \end{bmatrix}, \quad Q = \begin{bmatrix} 1 & 0 & 0 & -c_x \\ 0 & 1 & 0 & -c_y \\ 0 & 0 & 0 & f \\ 0 & 0 & 1/B & 0 \end{bmatrix}$$

The points sit on the image grid, so each 2×2 pixel block splits into two triangles; triangles that span a large depth jump are dropped. Faces take their colour from the texture or from a Lambertian shade $\max(0, \mathbf{n} \cdot \mathbf{I})$. The depth resolution is $\Delta Z = Z^2/(fB)$ per disparity pixel, and the Middlebury offset $d_{\min}$ has to be added before reprojection.

$$\Delta Z = \frac{Z^2}{fB} \Delta d$$

4. Algorithm / Procedure

1. Load the SGBM disparity (641×555) and the left image; recompute them if they are missing.
2. Build Q from $f = 1870$ px, $B = 160$ mm and the image centre; add $d_{\min} = 135$ px.

3. Apply `cv2.reprojectImageTo3D` to get (X, Y, Z) in millimetres and flip the Y axis.
4. Mask invalid disparities and the 1 % farthest points, attach RGB colours and write `aloe.ply`.
5. Sub-sample the grid every 4th pixel, form two triangles per cell (depth span under 50 mm) and shade them.
6. Render the point cloud from two viewpoints, the textured and shaded mesh, and the top and side views.

5. Output

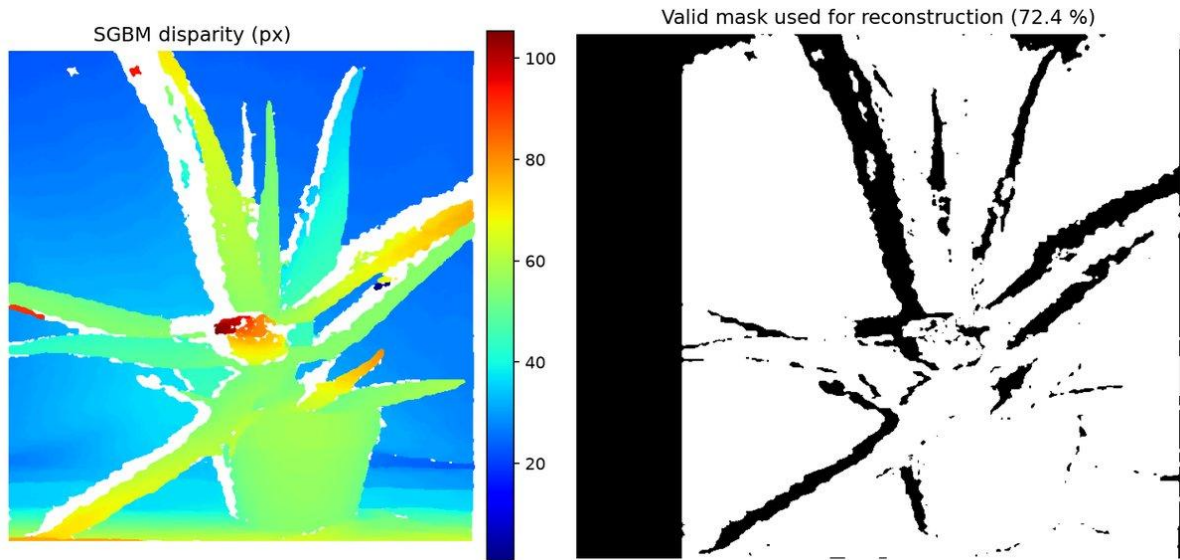


Figure 5.1: SGBM disparity map (left) and the mask of pixels used for the reconstruction (right).

3D point cloud reconstructed from the stereo pair (20 000 of 257493 points shown)

Coloured point cloud (elev 18, azim -65)

Coloured point cloud (elev 25, azim -120)

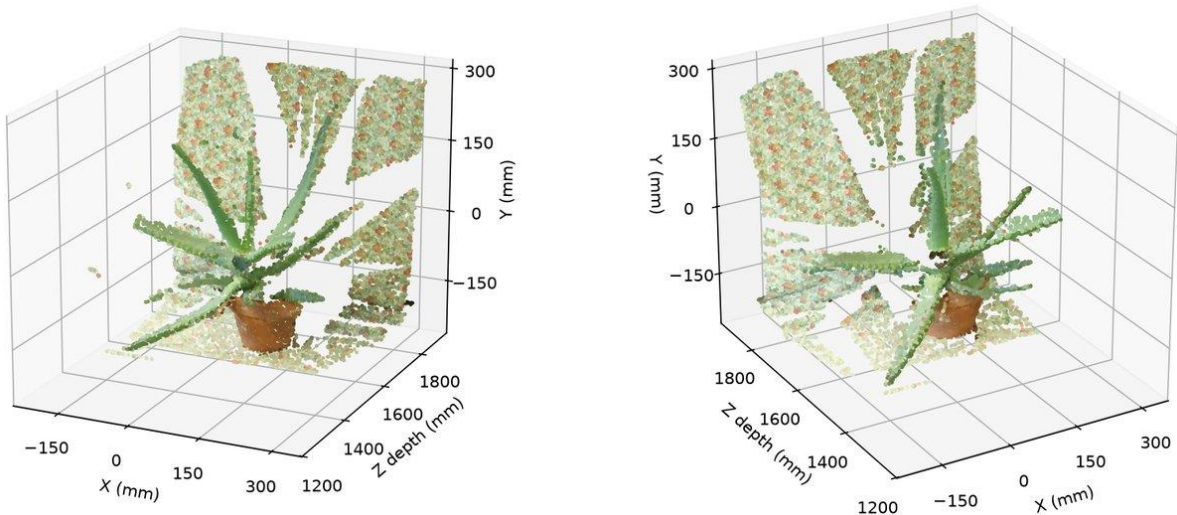
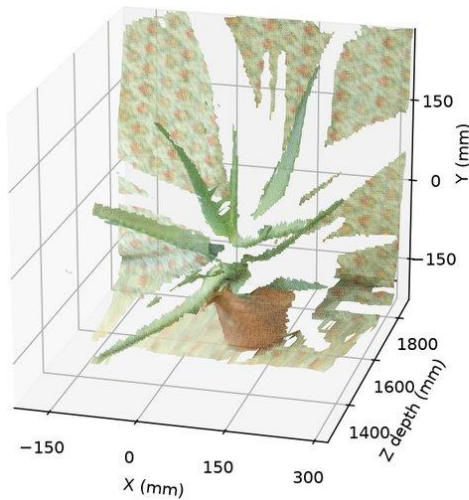


Figure 5.2: Coloured 3D point cloud of the Aloe scene from two viewpoints (20 000 of 257 493 points).

Surface mesh with 28461 triangles (depth jumps > 50 mm not connected)

Textured triangle mesh



Shaded mesh (Lambertian)

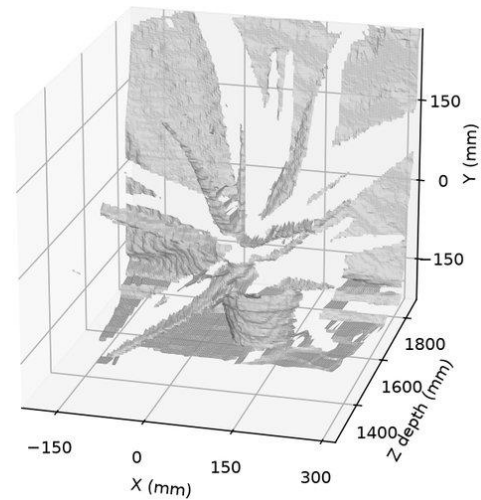
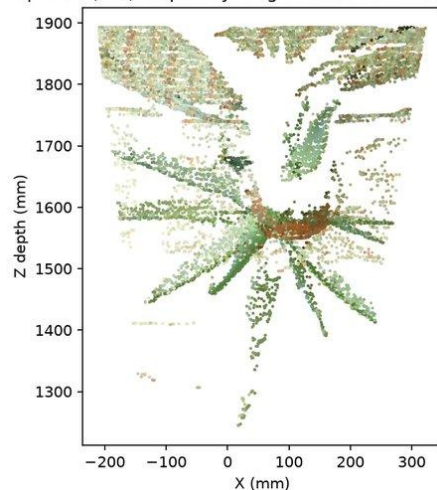


Figure 5.3: Grid triangle mesh (28 461 triangles), textured (left) and Lambertian shaded (right).

Top view (X-Z): depth layering of leaves and background



Side view (Z-Y)

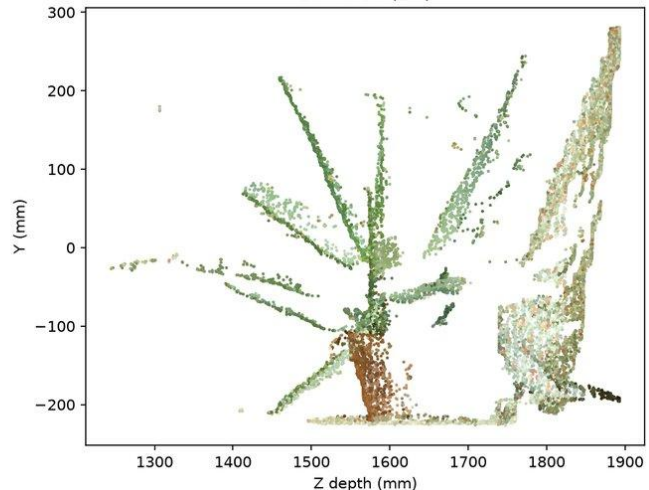


Figure 5.4: Orthographic top (XZ) and side (ZY) views; leaves, pot and background lie at different depths.

Console output

```
Image 641 x 555; valid 3D points: 257493 (72.4 % of pixels)
Depth range kept: 1245 .. 1893 mm, width span -211 .. 322 mm
Point cloud written: aloes.ply with 257493 coloured vertices
Mesh: 28461 triangles on a 139 x 161 vertex grid (step 4)
Figures written to E:\Plant-Disease-Detection-and-Analysis-System\LAB\outputs\exp05
```

6. Results and Observations

257 493 valid 3D points (72.4 % of the pixels) were exported to aloes.ply. They span 1245 to 1893 mm in depth and -211 to 322 mm in width.

The mesh has 28 461 triangles on a 139×161 grid. Dropping triangles with a depth span above 50 mm keeps the leaves separate from the background.

In the top and side views the individual leaves sit at different depths, the pot is at about 1.55 m and the background lies 0.2 to 0.6 m behind the plant. The depth steps of about 8 mm per disparity pixel are visible.

7. Conclusion

The reprojection matrix Q turns every matched pixel into a metric 3D point, and the image grid supplies the mesh connectivity. Occlusions leave holes and disparity quantisation leaves depth steps, but the plant, pot and background all appear at their measured distances.

Experiment No. 6

Implement Segmentation methods

Experiment No.	6
Title	Implement Segmentation methods
Date of experiment	
Tools / software used	Python 3.11, OpenCV 5.0, NumPy, Matplotlib
Input data	PlantVillage leaf images (grape black rot for thresholding, colour and clustering methods, potato late blight for GrabCut and region growing) and the smarties.png sample image from OpenCV for watershed object counting.
Program file	exp06_segmentation.py

1. Aim

To implement and compare segmentation by thresholding, colour clustering, region growing, marker-based watershed and GrabCut, and to apply these methods to leaf and lesion segmentation.

2. Objectives

- Segment images with global, Otsu and adaptive thresholding.
- Separate leaf and lesions with HSV colour ranges and cluster colours with K-means and mean shift.
- Count touching objects with watershed and extract a foreground with GrabCut and region growing.

3. Theory

Segmentation partitions an image into homogeneous regions. Global thresholding labels every pixel below a fixed T . Otsu chooses T by maximising the between-class variance $\sigma_b^2(T)$, and adaptive thresholding subtracts a constant C from the local mean, which copes with uneven lighting. In HSV space a range of hue, saturation and value selects one colour class, such as green leaf or brown lesion, and the ratio of lesion pixels to leaf pixels is a measure of disease severity.

$$\sigma_b^2(T) = w_0(T)w_1(T)[\mu_0(T) - \mu_1(T)]^2, \quad T^* = \arg\max_T \sigma_b^2(T)$$

K-means groups the pixel colours into k clusters by minimising the compactness J . Mean shift moves each pixel to the mode of the local colour density. Region growing starts from a seed and adds neighbours whose value differs from it by less than τ . The watershed transform floods the inverted distance transform from markers, which splits touching objects. GrabCut models the colours with Gaussian mixtures and finds a minimum graph cut, starting from a rectangle around the object.

$$J = \sum_{i=1}^k \sum_{\mathbf{p} \in C_i} \|\mathbf{p} - \mathbf{m}_i\|^2$$

4. Algorithm / Procedure

1. Read the grape leaf image, resize to 512×512 , convert to grayscale and HSV.

2. Apply global ($T = 100$), Otsu and adaptive mean and Gaussian thresholds (31×31 , $C = 5$).
3. Build the leaf mask with `cv2.inRange` (hue 15-95) and the brown lesion mask, then compute the lesion percentage.
4. Cluster RGB values with `cv2.kmeans` for $k = 2, 3, 4$; apply mean shift filtering and cluster again.
5. On the smarties image threshold the saturation, compute the distance transform, mark the peaks and run `cv2.watershed`.
6. Run GrabCut (rectangle, 5 iterations) on the potato leaf and grow a region from a seed on the Lab a-channel ($\tau = 14$).
7. Compute the IoU of the two masks and save all figures.

5. Output

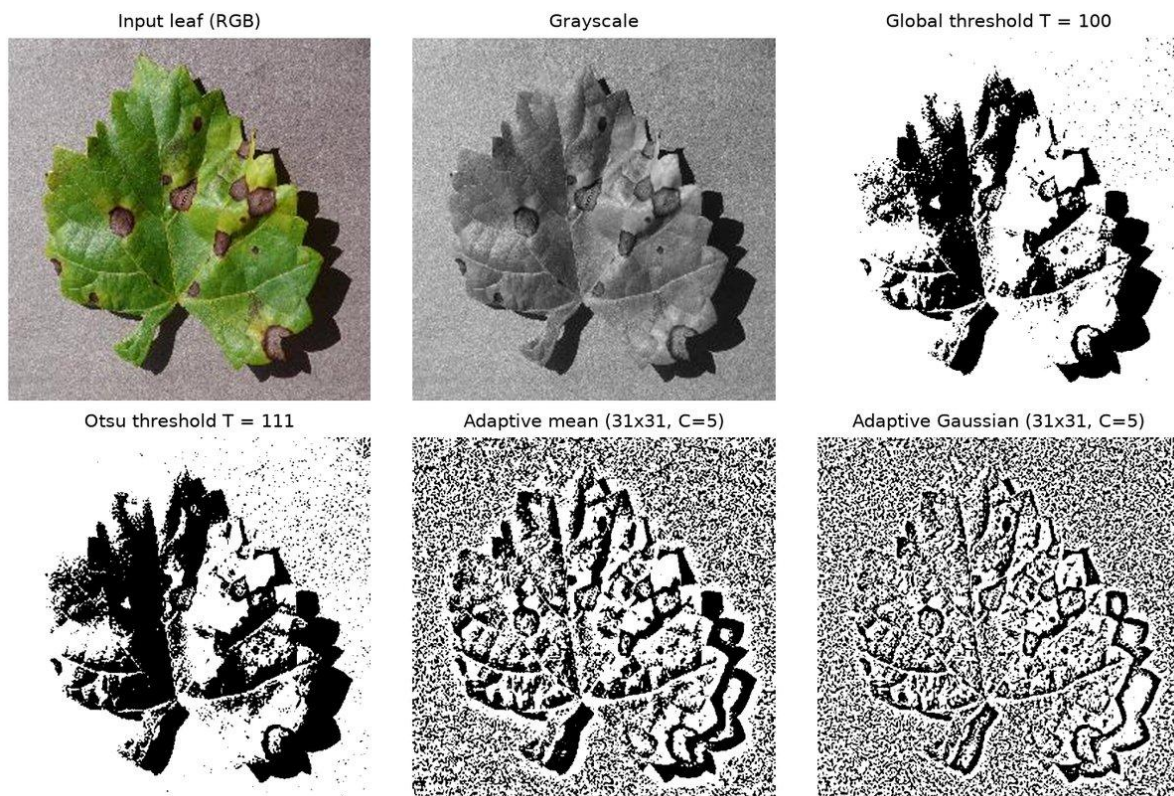


Figure 6.1: Global ($T = 100$), Otsu ($T = 111$) and adaptive thresholding of the grape leaf image.

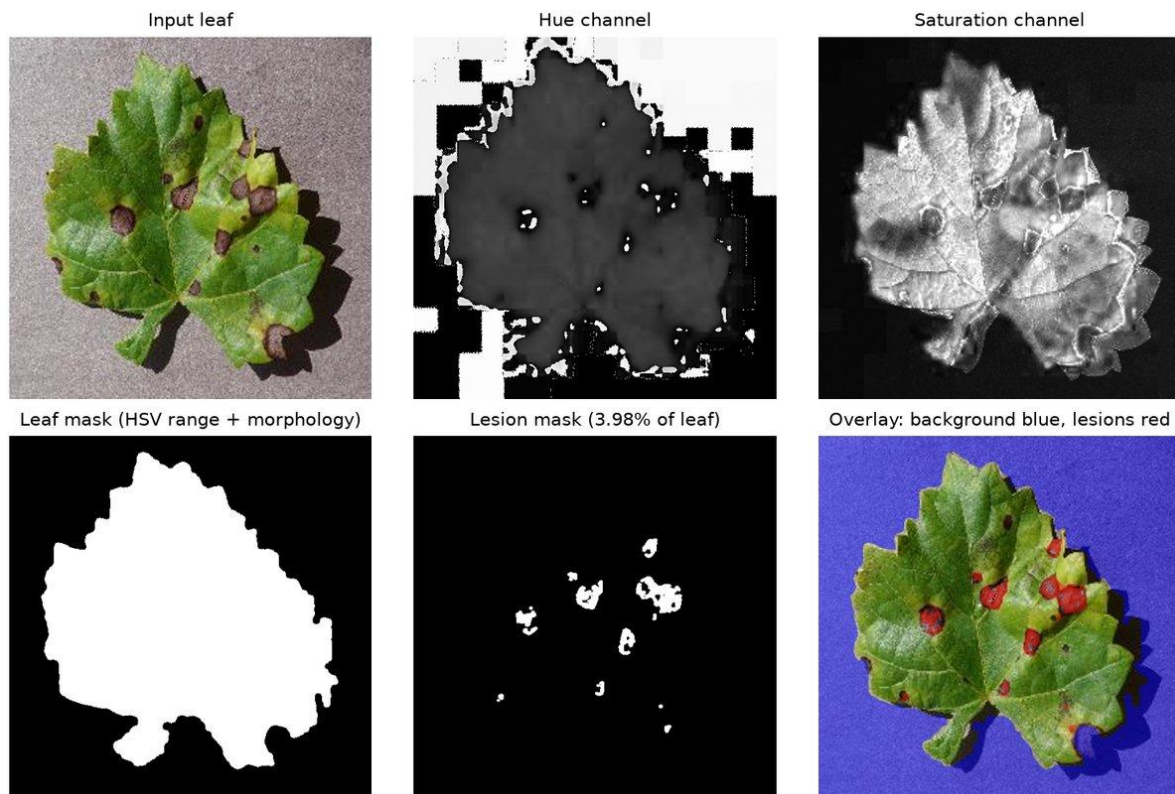


Figure 6.2: HSV segmentation: hue and saturation channels, leaf mask, lesion mask and overlay (lesions 3.98 percent of the leaf).

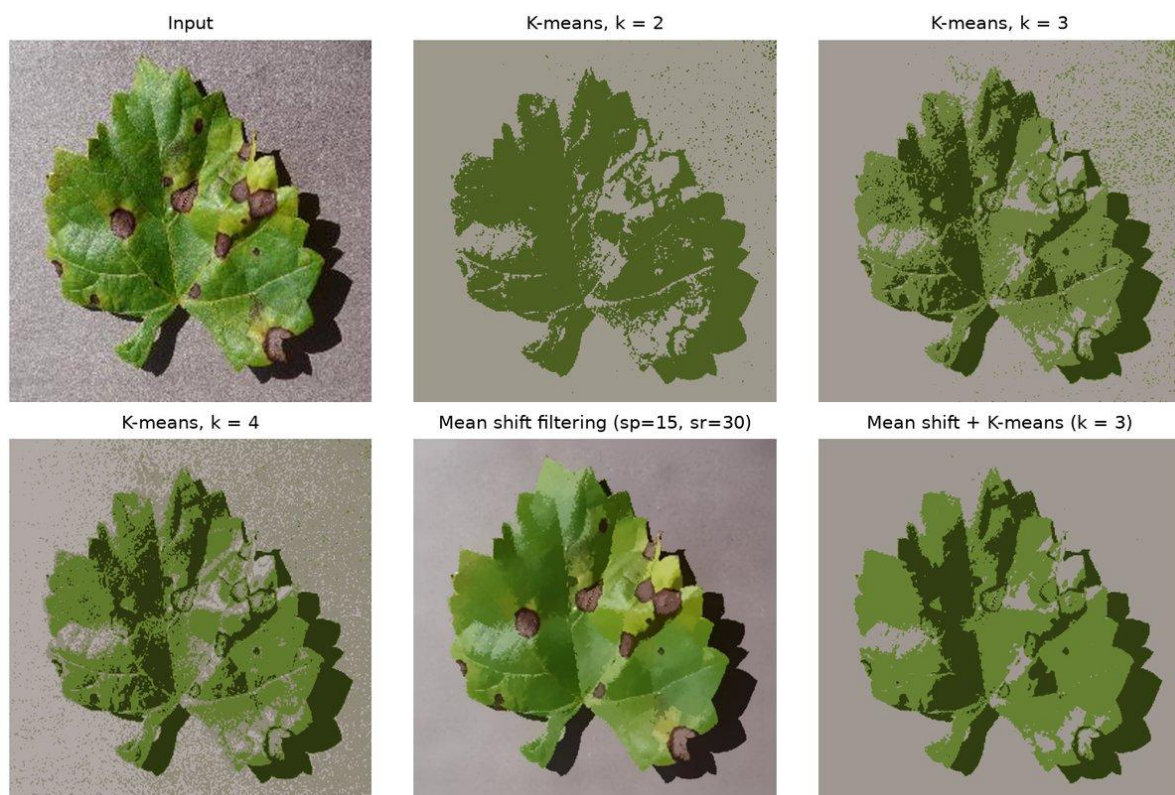


Figure 6.3: K-means clustering with $k = 2, 3, 4$, mean shift filtering and K-means on the mean shift output.

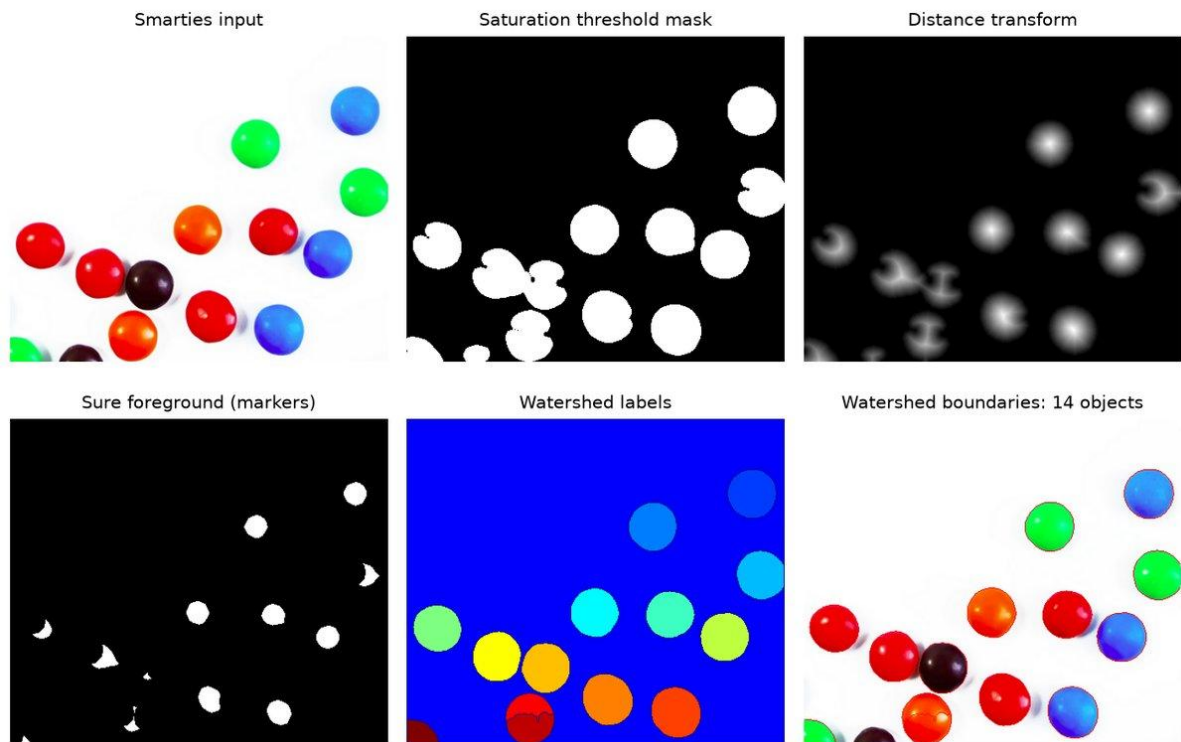


Figure 6.4: Marker based watershed on the smarties image: mask, distance transform, markers, labels and 14 segmented sweets.

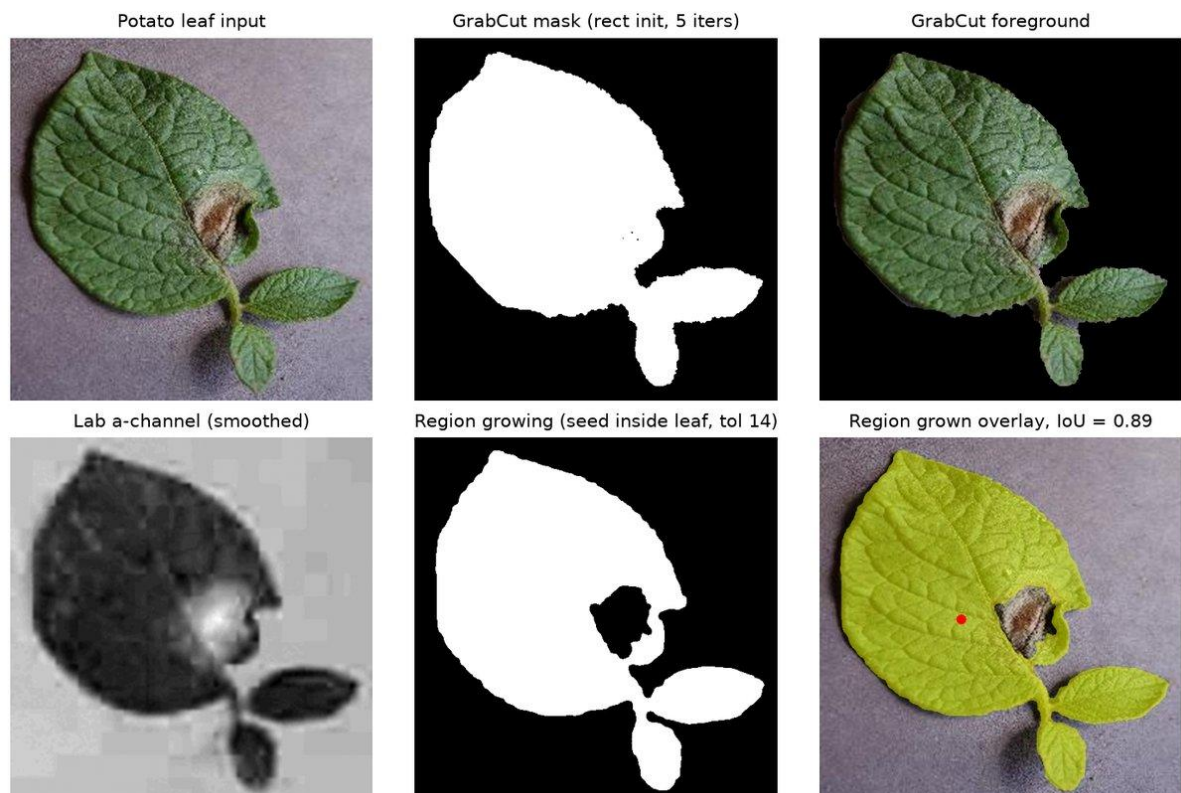


Figure 6.5: GrabCut extraction of a potato leaf and seeded region growing. The masks agree with $\mathrm{IoU} = 0.89$.

Console output

```
Otsu threshold selected automatically = 111
global T=100      foreground (dark) pixels = 23.9 %
Otsu              foreground (dark) pixels = 30.0 %
adaptive mean     foreground (dark) pixels = 39.8 %
adaptive Gaussian foreground (dark) pixels = 37.6 %
Leaf area = 118513 px (45.2 % of image), lesion area = 4719 px = 3.98 % of leaf, lesions counted = 11
K-means k=2 : compactness = 5.824e+08, cluster sizes = [163292, 98852]
K-means k=3 : compactness = 3.637e+08, cluster sizes = [71854, 142518, 47772]
K-means k=4 : compactness = 2.382e+08, cluster sizes = [61307, 78713, 82371, 39753]
Mean shift (sp=15, sr=30): unique colours 103847 -> 31851
Watershed on smarties: 14 objects segmented
GrabCut foreground = 42.9 % of image
Region growing from seed (256, 200) on Lab a-channel (tol 14): region = 40.2 % of image, IoU with
GrabCut = 0.886
Saved figures to outputs/exp06
```

6. Results and Observations

Otsu selected $T = 111$ (30.0 percent dark pixels). The fixed $T = 100$ gave 23.9 percent, and the adaptive methods 39.8 and 37.6 percent because they also respond to the background texture.

The HSV leaf mask covered 45.2 percent of the image. The black rot lesions occupy 3.98 percent of the leaf area in 11 separate lesions. K-means reduced J from 5.82×10^8 ($k = 2$) to 2.38×10^8 ($k = 4$).

Watershed separated 14 sweets, including the touching ones. GrabCut extracted the potato leaf as 42.9 percent of the image and region growing gave 40.2 percent; the IoU between the two masks is 0.886.

7. Conclusion

Thresholding needs a bimodal histogram, and on this leaf only the HSV colour ranges separated the leaf from its lesions and gave a severity measure. Region growing, watershed and GrabCut use connectivity and produced object masks without background speckle.

Experiment No. 7

Construct 3D model from defocus image

Experiment No.	7
Title	Construct 3D model from defocus image
Date of experiment	
Tools / software used	Python 3.11, OpenCV 5.0 (GaussianBlur, filter2D, Laplacian, boxFilter, medianBlur), NumPy, Matplotlib (mplot3d)
Input data	Synthetic focal stack of 15 slices (256 x 256 px) rendered from a PlantVillage tomato leaf image (class Tomato__Septoria_leaf_spot) with a known ground-truth depth map.
Program file	exp07_shape_from_defocus.py

1. Aim

To recover the depth, and hence the shape, of a scene from the defocus blur in images taken at different focus settings, and to build a textured 3D surface from the recovered depth map.

2. Objectives

- To understand how defocus blur encodes the distance from the plane of focus.
- To implement shape from focus with the SML and variance-of-Laplacian measures and sub-slice interpolation.
- To build an all-in-focus image and a textured 3D surface, and to try two-image depth from defocus.

3. Theory

A thin lens focuses a point at distance u onto the sensor at distance v when $1/u + 1/v = 1/F$. Points at other distances form a Gaussian blur whose σ grows with the distance from the focus plane, so blur carries depth.

$$\frac{1}{u} + \frac{1}{v} = \frac{1}{F}, \quad \sigma \propto \left| \frac{1}{u} - \frac{1}{u_0} \right|$$

Shape from focus captures a focal stack of N images at different focus settings. A focus measure such as the Sum-Modified-Laplacian (SML) or the variance of the Laplacian scores the local sharpness in every slice. The slice with the largest response gives each pixel's depth; a parabola fitted to the log of the three values around the peak refines it to a fraction of a slice. Copying each pixel from its sharpest slice gives an all-in-focus image. Depth from defocus needs only two images, because the ratio of their high-frequency energies depends on the relative blur.

$$\text{SML}(x, y) = \sum_{(i,j) \in W} (|2I(i, j) - I(i-1, j) - I(i+1, j)| + |2I(i, j) - I(i, j-1) - I(i, j+1)|)$$

4. Algorithm / Procedure

1. Load the leaf image, resize it to 256×256 and define a ground-truth depth (a tilted plane plus a Gaussian hill).
2. Render 15 slices by blurring each pixel with $\sigma = 0.7|d - k|$ and adding sensor noise.
3. Compute the SML and variance-of-Laplacian focus measures of every slice in 7×7 windows.
4. Take the slice of maximum response per pixel, refine it by log-parabolic interpolation, median filter and compare with the truth.
5. Build the all-in-focus image from the sharpest slice of every pixel.
6. Form the two-image ratio $E_0/(E_0 + E_{14})$ of Laplacian energies and correlate it with the true depth.
7. Plot the stack, the focus curves, the depth and error maps, the textured 3D surface and the defocus ratio.

5. Output

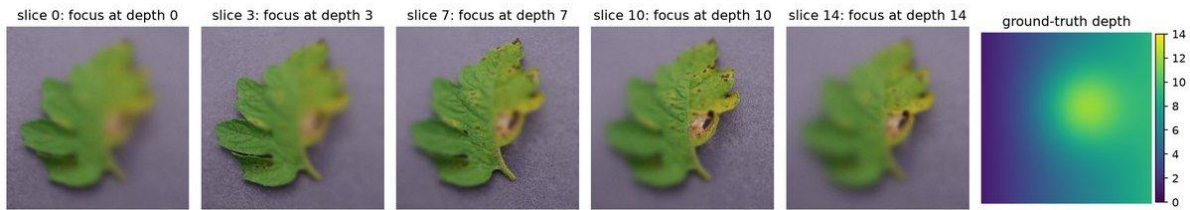


Figure 7.1: Five slices of the synthetic focal stack (focus at 0, 3, 7, 10 and 14) and the ground-truth depth map.

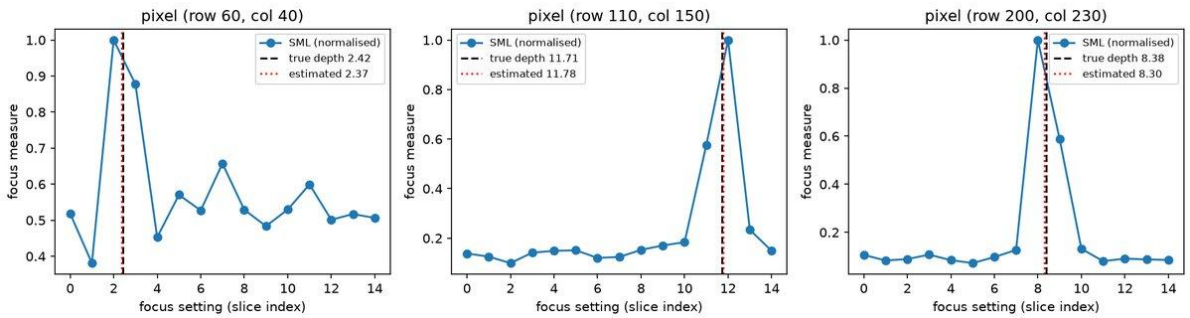


Figure 7.2: Normalised SML focus curves of three pixels. Dashed lines mark the true depth, dotted lines the interpolated estimate.

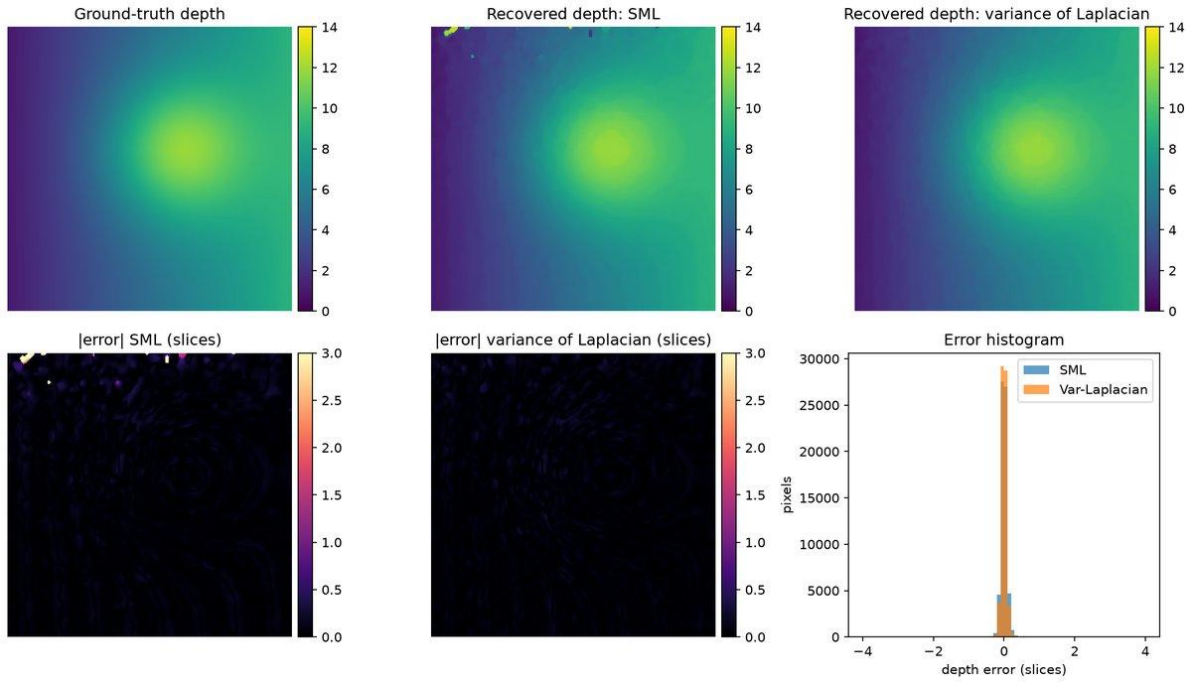


Figure 7.3: Ground-truth depth, the depths recovered with SML and with the variance of the Laplacian, absolute error maps and error histogram.

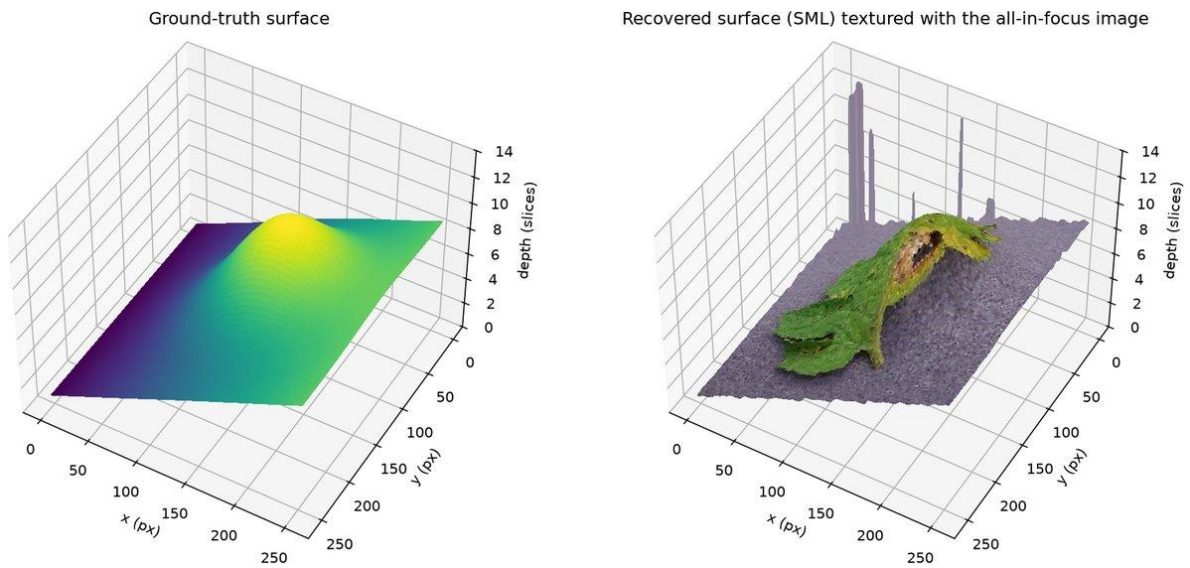


Figure 7.4: Ground-truth surface (left) and the recovered surface textured with the all-in-focus composite (right).

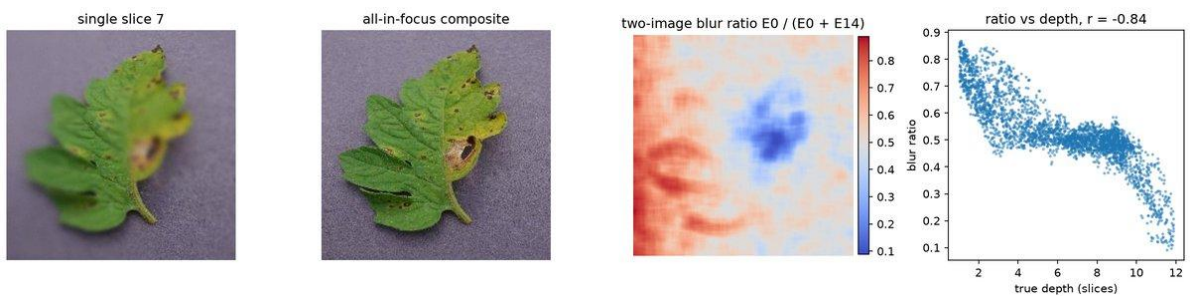


Figure 7.5: A single slice against the all-in-focus composite, the two-image blur ratio and its relation to true depth (correlation -0.84).

Console output

```
Texture: 002533c1-722b-44e5-9d2e-91f7747b2543_Keller.St_CG 1831.JPG; stack of 15 slices; ground-
truth depth range 1.00 .. 11.87
SML      : RMSE 0.349 slices, MAE 0.071 slices, pixels within 1 slice 99.8 %
Var-Laplacian : RMSE 0.066 slices, MAE 0.050 slices, pixels within 1 slice 100.0 %
All-in-focus image: mean SML 3.38 x that of the middle slice
Two-image defocus ratio vs true depth: correlation coefficient -0.838
Figures written to E:\Plant-Disease-Detection-and-Analysis-System\LAB\outputs\exp07
```

6. Results and Observations

SML recovered the depth with an RMSE of 0.349 and an MAE of 0.071 slices (99.8 % of the pixels within one slice). The variance of the Laplacian did better: RMSE 0.066 and MAE 0.050 slices, with every pixel within one slice.

The focus curves of three sample pixels peak at true depths of 2.42, 11.71 and 8.38 slices. The interpolated estimates were 2.37, 11.76 and 8.32 slices.

The all-in-focus composite has 3.38 times the mean SML of the middle slice. The two-image blur ratio correlates with the true depth with a coefficient of -0.838.

7. Conclusion

The position of maximum sharpness in a focal stack gives the depth of every textured pixel to a fraction of a slice, and together with the all-in-focus image it yields a textured 3D surface. The two-image defocus variant needs far fewer images but is less accurate.

Experiment No. 8

Construct 3D model from Images

Experiment No.	8
Title	Construct 3D model from Images
Date of experiment	
Tools / software used	Python 3.11, OpenCV 5.0 (SIFT, findEssentialMat, recoverPose, triangulatePoints, solvePnPRansac), NumPy, Matplotlib
Input data	Three colour photographs (1024 x 768) 001.jpg, 002.jpg and 003.jpg of the Wadham College courtyard from the Oxford Visual Geometry Group multi-view data set (robots.ox.ac.uk/~vgg/data/mview); the supplied 3 x 4 camera matrices are used only to obtain K and to check the estimated poses.
Program file	exp08_sfm_from_images.py

1. Aim

To reconstruct a sparse, coloured 3D model of a scene from ordinary photographs taken from different viewpoints, using feature matching, pose estimation from the essential matrix, triangulation and the registration of further views.

2. Objectives

- Match SIFT features between photographs and reject wrong matches with the ratio test and RANSAC.
- Recover the relative pose from the essential matrix E , triangulate the matches and add a third view with PnP.
- Evaluate the model by its reprojection error and by comparison with the supplied camera matrices.

3. Theory

Structure from motion recovers the 3D scene and the camera poses from overlapping images. Features are matched, the relative camera geometry is estimated from the matches, and the matched points are triangulated. SIFT descriptors are matched by Euclidean distance and kept only when they pass Lowe's ratio test $d_1/d_2 < 0.8$.

Two calibrated views are related by the essential matrix $E = [t]_{\times} R$ through the epipolar constraint. RANSAC estimates E from the matches (cv2.findEssentialMat), and cv2.recoverPose decomposes it into R and t up to scale.

$$\mathbf{x}_2^T E \mathbf{x}_1 = 0, \quad E = [t]_{\times} R = \begin{bmatrix} 0 & -t_z & t_y \\ t_z & 0 & -t_x \\ -t_y & t_x & 0 \end{bmatrix} R$$

With $P_1 = K[I|0]$ and $P_2 = K[R|t]$, each match gives a linear system whose null vector is the 3D point (cv2.triangulatePoints). A point is accepted when its reprojection error $e = \|\mathbf{x} - \hat{\mathbf{x}}\|$ is small. Further views are registered with PnP on the reconstructed points and add new triangulations.

$$\begin{bmatrix} u_1 \mathbf{p}_1^{3T} - \mathbf{p}_1^{1T} \\ v_1 \mathbf{p}_1^{3T} - \mathbf{p}_1^{2T} \\ u_2 \mathbf{p}_2^{3T} - \mathbf{p}_2^{1T} \\ v_2 \mathbf{p}_2^{3T} - \mathbf{p}_2^{2T} \end{bmatrix} \mathbf{X} = 0$$

4. Algorithm / Procedure

1. Load three images and the supplied projection matrices, and decompose them into a shared K and ground-truth poses.
2. Detect up to 8000 SIFT keypoints per image and match views 1-2 with a brute-force matcher and a ratio test of 0.8.
3. Estimate E with `cv2.findEssentialMat` (RANSAC), recover R and t with `cv2.recoverPose` and scale t to the known baseline.
4. Triangulate the inliers from P_1 and P_2 and keep the points with positive depth and a reprojection error below 2 px.
5. Register view 3 with `cv2.solvePnP` on its matches to known 3D points and triangulate its remaining matches.
6. Remove points with parallax below 1° or outside the 1-99 percentile box and save a coloured PLY file.
7. Draw the inlier matches, the epipolar lines, two views of the point cloud with the cameras, and the error histogram.

5. Output

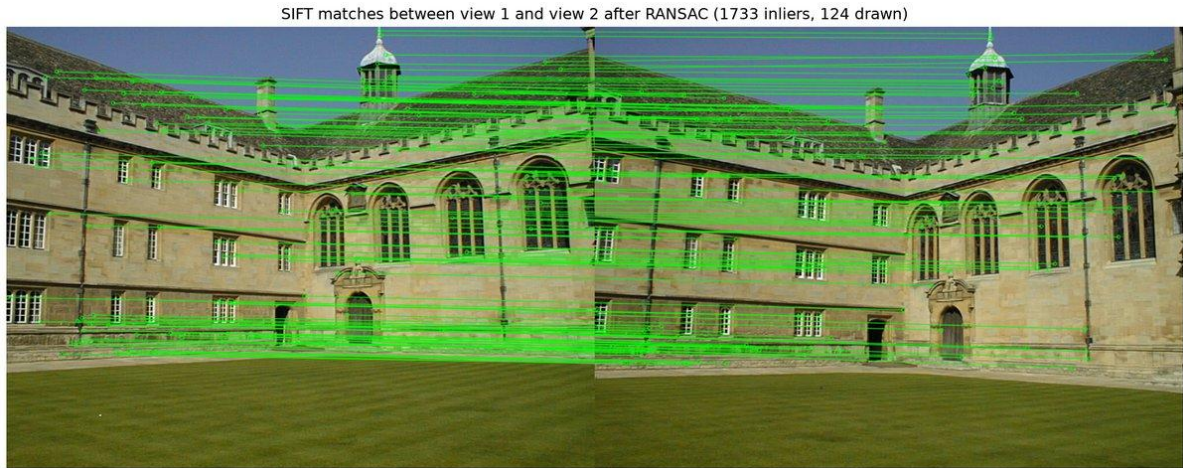


Figure 8.1: SIFT matches between views 1 and 2 that passed the ratio test and essential-matrix RANSAC (124 of 1733 drawn).

Epipolar geometry from the estimated essential matrix ($F = K^{-T} E K^{-1}$)

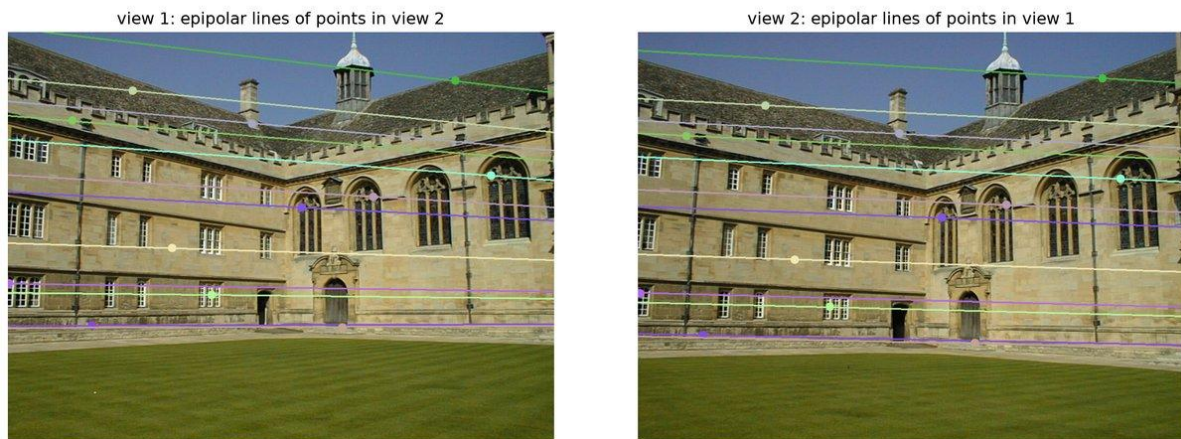


Figure 8.2: Epipolar lines from the estimated E for twelve inlier points; each point lies on the line of its match.

Sparse 3D model: 2383 points from 3 views (red triangles: estimated cameras)

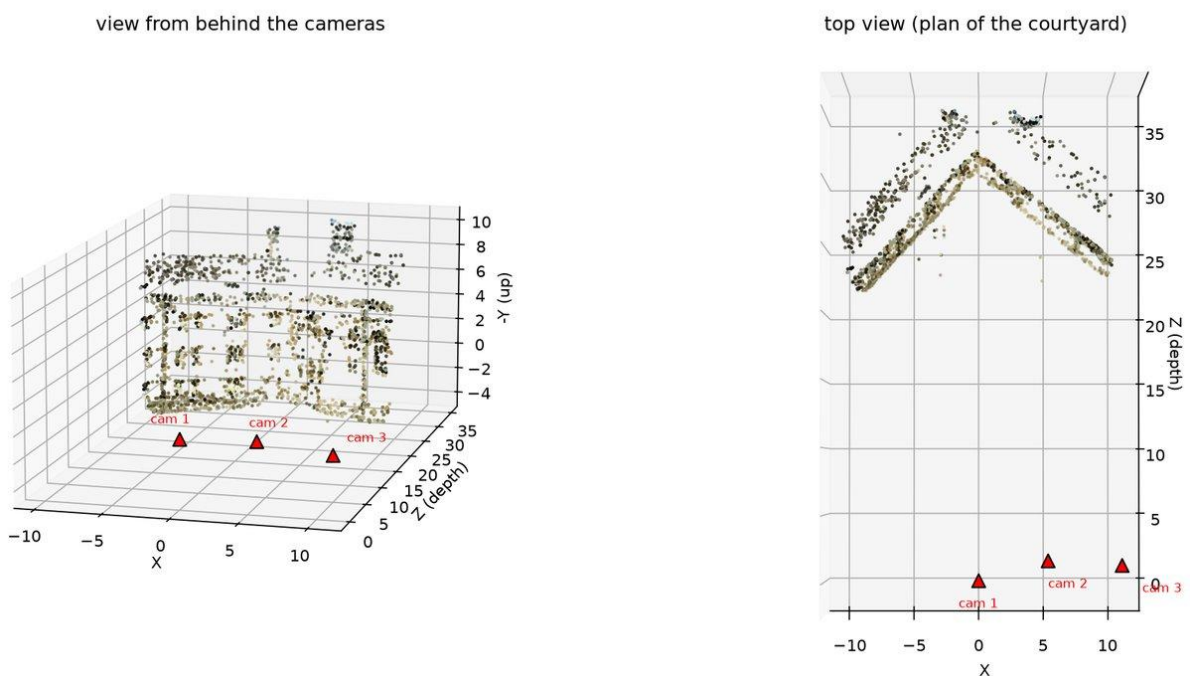


Figure 8.3: The sparse model of 2383 coloured points with the three camera positions, in oblique and plan views.

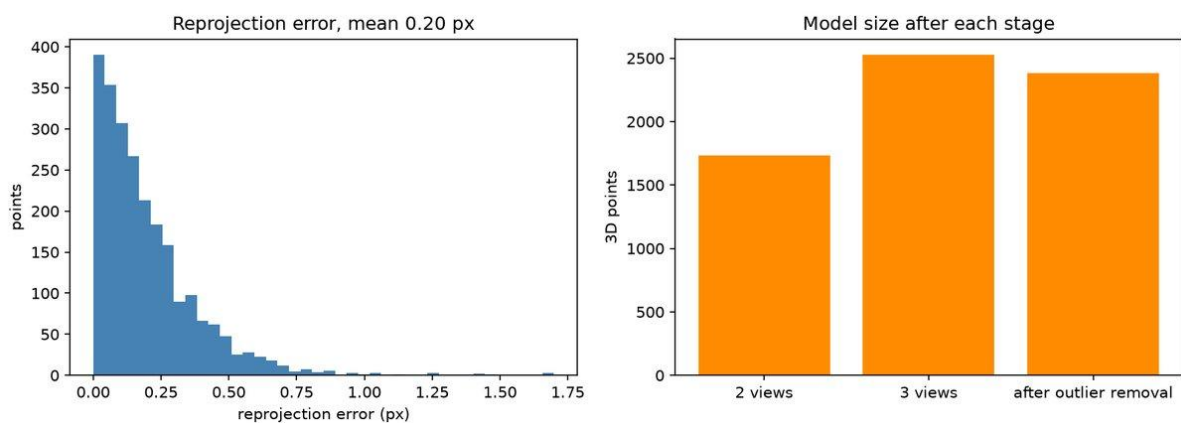


Figure 8.4: Reprojection error histogram (mean 0.20 px) and model size after each reconstruction stage.

Console output

```

Images: 3 views of 1024x768 px. Shared intrinsic matrix K:
[[1147.8439    0.    526.3003]
 [    0.    1139.4994 391.9789]
 [    0.    0.    1.    ]]
SIFT keypoints per image: [5998, 5872, 5956]
View 1-2: 2107 ratio-test matches, 1735 essential-matrix inliers, 1733 pass the cheirality check
Triangulated 1733 points from 2 views (mean reprojection error 0.164 px)
Pose 1->2 vs ground truth: rotation error 0.19 deg, translation direction error 1.07 deg
View 3: 1905 matches, 858 have known 3D points, PnP inliers 730; pose 2->3 error: rotation 0.24 deg,
translation direction 3.36 deg
    added 792 new points -> total 2525
Removed 142 outlier points (parallax < 1 deg or outside the 1-99 percentile box)
Final model: 2383 coloured 3D points, mean reprojection error 0.199 px, median 0.148 px, median
parallax 10.85 deg
Saved points.ply and figures to outputs/exp08

```

6. Results and Observations

SIFT found 5998, 5872 and 5956 keypoints in the three images. Of the 2107 ratio-test matches between views 1 and 2, 1735 were essential-matrix inliers, and their triangulation had a mean reprojection error of 0.164 px.

The estimated pose differs from the ground truth by 0.19° in rotation and 1.07° in translation direction. View 3, registered with 730 PnP inliers, differs by 0.24° and 3.36° and added 792 points.

After 142 outliers were removed the model holds 2383 coloured points, with a mean reprojection error of 0.199 px and a median parallax of 10.85° . The plan view shows the two wings of the courtyard.

7. Conclusion

Three photographs were enough to reconstruct a sparse 3D model of a real building, with camera poses within a fraction of a degree of the ground truth. The model is known only up to scale; bundle adjustment and dense matching would complete it.

Experiment No. 9

Implement optical flow method

Experiment No.	9
Title	Implement optical flow method
Date of experiment	
Tools / software used	Python 3.11, OpenCV 5.0 (calcOpticalFlowPyrLK, goodFeaturesToTrack, calcOpticalFlowFarneback, DISOpticalFlow), NumPy, Matplotlib
Input data	OpenCV sample videos vtest.avi (static surveillance camera, 768 x 576, 10 fps, walking pedestrians) and Megamind.avi (animation clip, 720 x 528) from the OpenCV samples/data repository.
Program file	exp09_optical_flow.py

1. Aim

To estimate the apparent motion of image points between consecutive video frames with sparse (Lucas-Kanade) and dense (Farneback, DIS) optical flow, and to check the Lucas-Kanade solution against an implementation written from scratch.

2. Objectives

- Track Shi-Tomasi corners through a video with pyramidal Lucas-Kanade and draw their tracks.
- Compute dense Farneback and DIS optical flow and show it as a colour-wheel image and a vector field.
- Solve the Lucas-Kanade least-squares problem in NumPy and compare the answer with OpenCV.

3. Theory

Optical flow is the 2D vector field (u, v) of the apparent motion of each pixel between consecutive frames. Assuming constant brightness, a first-order Taylor expansion gives the flow constraint, one equation in two unknowns (aperture problem):

$$I(x, y, t) = I(x + u, y + v, t + 1) \Rightarrow I_x u + I_y v + I_t = 0$$

Lucas-Kanade assumes constant flow inside a small window (21×21 pixels here). Stacking the window's constraints gives $A\mathbf{v} = -\mathbf{b}$ with $A = [I_x I_y]$ and $\mathbf{b} = I_t$, solved by least squares. $A^T A$ is the structure tensor, well conditioned only at corners, so tracking starts from Shi-Tomasi corners; a pyramid handles large motions.

$$\mathbf{v} = \begin{bmatrix} u \\ v \end{bmatrix} = (A^T A)^{-1} A^T (-\mathbf{b}), \quad A^T A = \begin{bmatrix} \sum I_x^2 & \sum I_x I_y \\ \sum I_x I_y & \sum I_y^2 \end{bmatrix}$$

Dense methods return a vector at every pixel. Farneback fits a quadratic polynomial to each neighbourhood; DIS matches patches on a coarse grid and refines variationally, several times faster. The field is shown as a quiver plot or an HSV image (hue = direction, brightness = magnitude).

4. Algorithm / Procedure

1. Read the first 61 frames of vtest.avi in grayscale and detect up to 400 Shi-Tomasi corners in frame 0.
2. Track the corners with cv2.calcOpticalFlowPyrLK (21 × 21 window, 3 levels) and draw the tracks of points that move more than 0.5 px.
3. Every 15 frames save a snapshot and re-detect corners in regions that have none.
4. Compute Farneback and DIS (medium) flow on 20 consecutive frame pairs and record the run time and flow magnitudes.
5. Show the flow as an HSV colour-wheel image and a 16 px quiver plot, then repeat on a Megamind.avi pair.
6. Solve Lucas-Kanade in NumPy from the normal equations $A^T A \mathbf{v} = -A^T I_t$ with iterative refinement.
7. Run OpenCV single-level LK on the same ten corners and compare the two results.

5. Output

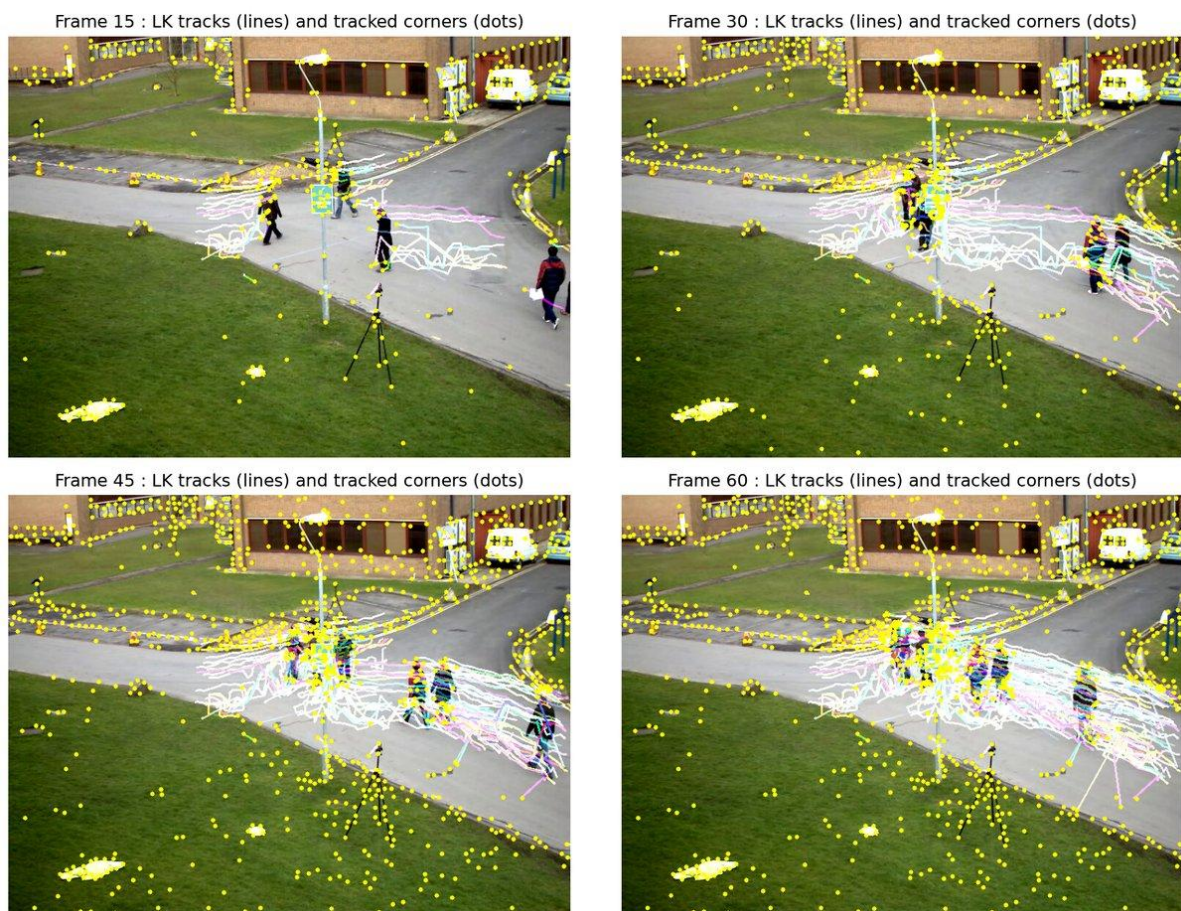


Figure 9.1: Sparse Lucas-Kanade tracks on vtest.avi at frames 15, 30, 45 and 60. Coloured lines are the paths of moving corners.

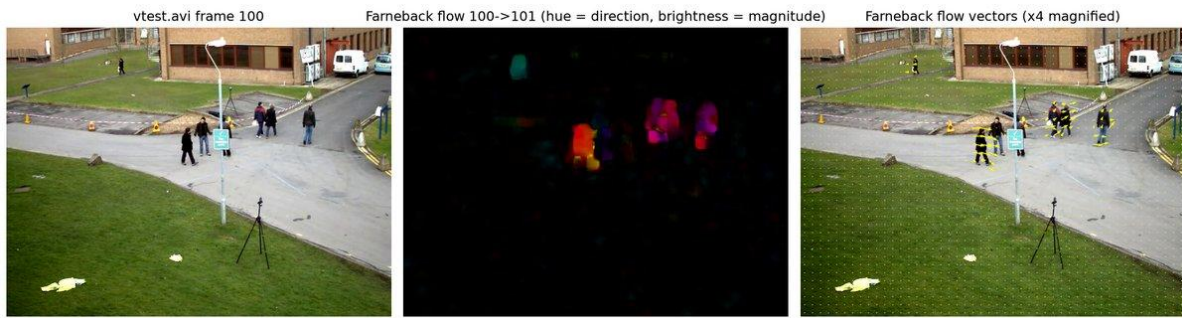


Figure 9.2: Farneback flow between frames 100 and 101 of vtest.avi: input, HSV colour-wheel image and flow vectors magnified four times.

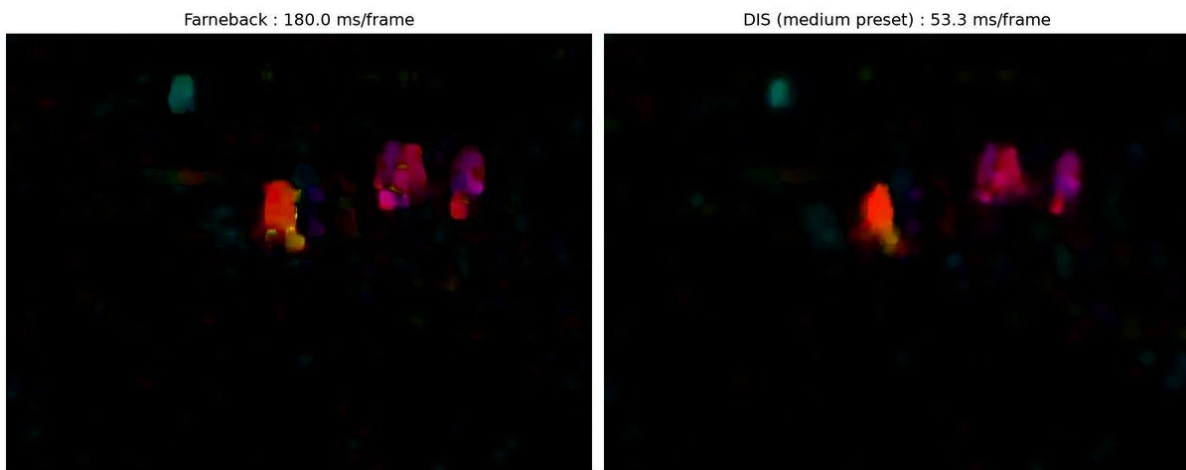


Figure 9.3: Farneback (left) and DIS medium (right) flow for the same pair, with run time per frame.



Figure 9.4: Farneback flow on Megamind.avi frames 121 to 122. The moving head gets one hue and the quiver plot shows its vectors.



Figure 9.5: NumPy Lucas-Kanade (red) and OpenCV single-level LK (cyan) on ten corners, arrows magnified six times.

Console output

```
Part A : sparse LK on vtest.avi frames 0-60 (initial corners = 400)
frame 0-> 1 : tracked=400 mean disp=0.61 px moving(>0.5px)=41 mean disp of moving=5.64 px
frame 14->15 : tracked=400 mean disp=0.75 px moving(>0.5px)=60 mean disp of moving=4.91 px
frame 29->30 : tracked=720 mean disp=0.39 px moving(>0.5px)=71 mean disp of moving=3.68 px
frame 44->45 : tracked=1013 mean disp=0.36 px moving(>0.5px)=116 mean disp of moving=2.86 px
frame 59->60 : tracked=1078 mean disp=1.14 px moving(>0.5px)=146 mean disp of moving=8.26 px
average over 60 frames : tracked=802.8 mean disp=0.741 px moving=102.2 moving disp=5.55 px

Part B : dense flow on 20 consecutive vtest pairs (frames 100-120, 768x576)
Farneback : 180.0 ms/frame mean |flow| = 0.231 px max |flow| = 13.74 px
DIS : 53.3 ms/frame mean |flow| = 0.169 px max |flow| = 12.46 px
Megamind pair 121->122 : mean |flow| = 0.64 px, max |flow| = 10.30 px

Part C : NumPy LK vs OpenCV LK (single level, 21x21 window) on 10 points of the pair 100->101
  x      y      NumPy (dx, dy)      OpenCV (dx, dy)      diff px
514.0 188.0      3.70 -2.10      3.70 -2.10      0.004
361.0 248.0      4.99 1.48      4.99 1.48      0.001
224.0 87.0      -2.18 0.19      -2.18 0.19      0.006
229.0 66.0      -2.34 0.53      -2.34 0.54      0.002
497.0 184.0      1.73 -0.91      1.74 -0.91      0.002
... (output truncated) ...
```

6. Results and Observations

Sparse LK kept all 400 initial corners. With re-detection the count grew to 1078 by frame 60, an average of 802.8 per frame. About 102 points per frame moved more than 0.5 px, with a mean displacement of 5.55 px; these are the pedestrians. Over all points the mean displacement was 0.74 px.

On 20 pairs of 768×576 frames Farneback took 180.0 ms per frame (mean 0.231 px, max 13.74 px). DIS took 53.3 ms (mean 0.169 px, max 12.46 px) and left a cleaner background. On the Megamind pair the mean flow was 0.64 px and the maximum 10.30 px.

The NumPy Lucas-Kanade solver agreed with OpenCV on all ten points. On a walking person both gave $(3.70, -2.10)$ px and on the static lamp post $(-0.01, 0.00)$ px. The mean absolute difference was 0.002 px.

7. Conclusion

Sparse Lucas-Kanade tracking recovers sub-pixel motion at textured corners, and both dense methods separate the walking people from the static background, DIS about three times faster than Farneback. The least-squares LK solution written by hand reproduced OpenCV to within 0.002 px.

Experiment No. 10

Implement object detection and tracking from video

Experiment No.	10
Title	Implement object detection and tracking from video
Date of experiment	
Tools / software used	Python 3.11, PyTorch 2.10 + torchvision 0.25 (fasterrcnn_resnet50_fpn_v2, COCO weights, CUDA GPU), OpenCV 5.0 (KalmanFilter, TrackerMIL, VideoWriter), SciPy (linear_sum_assignment), NumPy, Matplotlib
Input data	OpenCV sample video vtest.avi (768 x 576, 10 fps, fixed surveillance camera with walking pedestrians); the first 151 frames are processed. Detector weights: torchvision Faster R-CNN ResNet-50 FPN v2 trained on COCO 2017.
Program file	exp10_detection_tracking.py

1. Aim

To detect people in every frame of a surveillance video with a deep convolutional detector and to link the detections over time into object tracks with a Kalman filter based multi-object tracker.

2. Objectives

- Apply a pre-trained Faster R-CNN detector to video frames and measure its speed and detection count.
- Write a SORT-style tracker (one Kalman filter per object, Hungarian association on IoU) that keeps persistent IDs.
- Compare a model-free single-object tracker (OpenCV MIL) with tracking-by-detection.

3. Theory

Object detection returns a bounding box and a score for every object of a class. Tracking links the detections over time so that each object keeps one identity. Faster R-CNN is a two-stage detector. A ResNet-50 FPN backbone computes features, a Region Proposal Network proposes boxes, and a second head classifies and refines them. Only the COCO class person with a score above 0.6 is kept.

In tracking-by-detection (SORT) each track has a state $\mathbf{x} = (c_x, c_y, w, h, v_x, v_y)$. A constant-velocity Kalman filter predicts the state and the matched detection corrects it through the Kalman gain:

$$\hat{\mathbf{x}}_k^- = F\hat{\mathbf{x}}_{k-1}, \quad P_k^- = FP_{k-1}F^T + Q, \quad K = P_k^- H^T (HP_k^- H^T + R)^{-1}, \quad \hat{\mathbf{x}}_k = \hat{\mathbf{x}}_k^- + K(\mathbf{z}_k - H\hat{\mathbf{x}}_k^-)$$

Predictions and detections are matched by the Hungarian algorithm on an IoU cost. Matches below $\text{IoU} = 0.3$ are rejected. A track is confirmed after two hits and deleted after five misses. Model-free trackers such as MIL need only an initial box, but they can drift after an occlusion.

$$\text{IoU}(A, B) = \frac{|A \cap B|}{|A \cup B|}, \quad \text{cost}_{ij} = 1 - \text{IoU}(\hat{\mathbf{b}}_i, \mathbf{z}_j)$$

4. Algorithm / Procedure

1. Load the COCO pre-trained Faster R-CNN ResNet-50 FPN v2 model on the GPU with a score threshold of 0.6.
2. Run the detector on 151 frames of vtest.avi, keep the person boxes and record the inference time.
3. Predict every track with its constant-velocity Kalman filter.
4. Build the IoU matrix between predictions and detections and solve the assignment with the Hungarian algorithm ($\text{IoU} \geq 0.3$).
5. Update the matched tracks, start tentative tracks for unmatched detections (confirmed after 2 hits) and delete tracks after 5 misses.
6. Draw IDs, boxes and 40-frame trails and write the annotated video.
7. Initialise cv2.TrackerMIL on one person in frame 0, run it for 60 frames and plot its IoU with the detector box.

5. Output



Figure 10.1: Faster R-CNN person detections with confidence scores on frames 0, 50, 100 and 150 of vtest.avi.



Figure 10.2: Kalman + Hungarian multi-object tracker at frames 25 to 150. Each confirmed track keeps its colour, ID and a 40-frame trail.

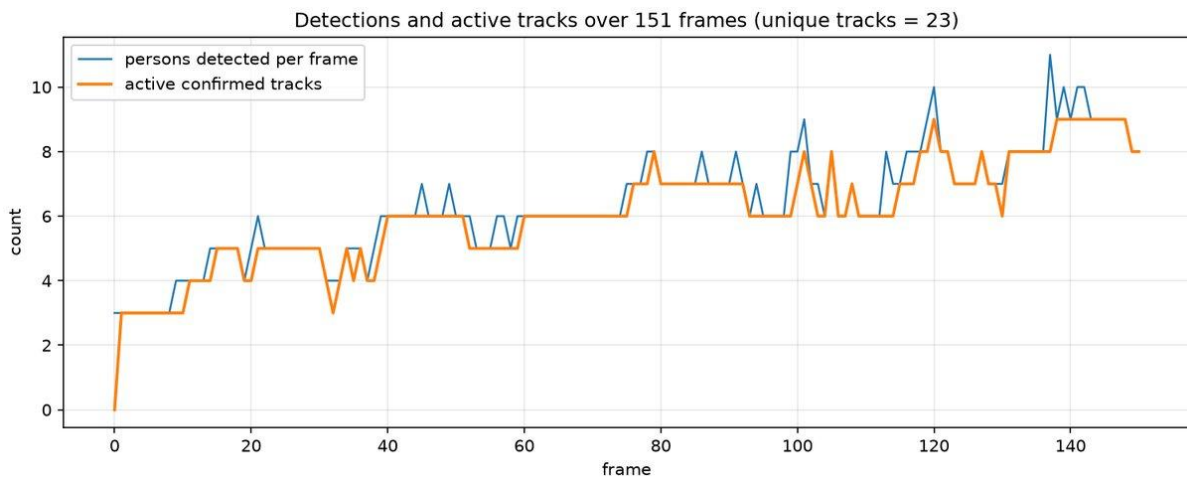


Figure 10.3: Persons detected per frame and active confirmed tracks over 151 frames. The track count follows the detections with a short delay.

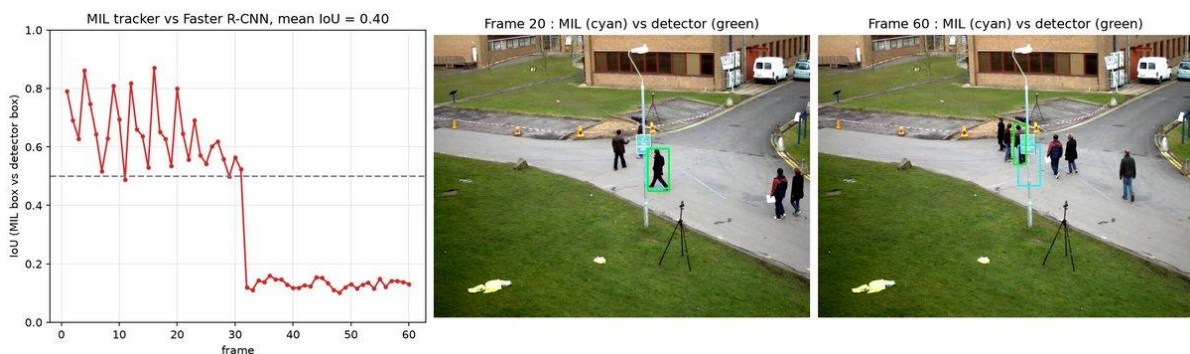


Figure 10.4: MIL tracker (cyan) and the detector box (green). The IoU falls to about 0.13 after the person passes the sign board.

Console output

```

Detector : Faster R-CNN ResNet-50 FPN v2 (COCO) on cuda, 151 frames of vtest.avi
mean inference time = 163.9 ms/frame (excluding warm-up frame), mean persons detected per frame =
6.36
Tracker : Kalman + Hungarian (IoU >= 0.3, confirm after 2 hits, delete after 5 misses)
unique confirmed tracks = 23, mean active tracks per frame = 6.08, max = 9
MIL tracker vs detector on one person over 60 frames : mean IoU = 0.397, min IoU = 0.102, frames with
IoU < 0.5 = 31
Outputs written to E:\Plant-Disease-Detection-and-Analysis-System\LAB\outputs\exp10

```

6. Results and Observations

Faster R-CNN processed the 768×576 frames on the RTX 3050 in 163.9 ms per frame. It found 6.36 persons per frame with scores of 0.99 to 1.00, and it also picked up the small distant pedestrians.

The Kalman + Hungarian tracker produced 23 unique confirmed tracks over 151 frames, with 6.08 active tracks per frame on average and a maximum of 9. Occlusions behind the lamp post occasionally split a track into a new ID.

The MIL tracker reached a mean IoU of 0.397 against the detector box, with a minimum of 0.102. It stayed on the person while the person was visible (IoU 0.5 to 0.87 for 31 frames) and then drifted onto the sign board after an occlusion.

7. Conclusion

The deep detector finds the people in every frame, and the Kalman filter with Hungarian association joins those detections into trajectories with stable IDs. MIL loses the person after an occlusion because it only has its initial box, whereas tracking-by-detection recovers because every frame is detected afresh.

Experiment No. 11

Face detection and Recognition

Experiment No.	11
Title	Face detection and Recognition
Date of experiment	
Tools / software used	Python 3.11, OpenCV 4.11 contrib build (CascadeClassifier, FaceDetectorYN, FaceRecognizerSF), NumPy, scikit-learn, Matplotlib
Input data	lena.jpg and messi5.jpg from the OpenCV samples, the Olivetti (AT&T) face database of 400 grayscale 64 x 64 images of 40 people fetched with scikit-learn, and the YuNet and SFace ONNX models from the OpenCV model zoo.
Program file	exp11_face_detection_recognition.py

1. Aim

To detect faces with the Viola-Jones Haar cascade and the YuNet convolutional detector, and to recognise faces with Eigenfaces (PCA), Local Binary Pattern histograms and the SFace embedding model.

2. Objectives

- Detect faces with a Haar cascade and with the YuNet CNN detector.
- Recognise Olivetti faces with Eigenfaces (PCA + SVM / 1-NN) and LBP histograms.
- Verify face pairs with SFace embeddings and cosine similarity.

3. Theory

Face detection locates faces in an image. Recognition then identifies or verifies the person. The Viola-Jones detector evaluates Haar-like features from the integral image with an AdaBoost cascade, which rejects most background windows in its first stages. YuNet is a small CNN that predicts boxes, scores and five landmarks in one pass and copes better with pose and illumination changes.

Eigenfaces treats a 64×64 face as a 4096-dimensional vector and projects it onto the k principal components of the training faces. An SVM or a nearest-neighbour rule classifies the projected vector. Local Binary Patterns encode each pixel by comparing it with its 8 neighbours; the histograms of the cells are concatenated and compared with the chi-square distance. SFace maps an aligned face to a 128-dimensional embedding, and two faces count as the same person when the cosine similarity exceeds 0.363.

$$\mathbf{w} = U_k^T(\mathbf{x} - \bar{\mathbf{x}}), \quad \chi^2(h_1, h_2) = \sum_i \frac{(h_{1,i} - h_{2,i})^2}{h_{1,i} + h_{2,i}}$$

$$\cos(\mathbf{e}_1, \mathbf{e}_2) = \frac{\mathbf{e}_1 \cdot \mathbf{e}_2}{\|\mathbf{e}_1\| \|\mathbf{e}_2\|} \geq 0.363$$

4. Algorithm / Procedure

1. Load the Haar face and eye cascades and create the YuNet detector (score threshold 0.7).
2. Detect faces in lena.jpg, messi5.jpg and a 12-face Olivetti collage with both detectors and draw boxes and landmarks.
3. Split Olivetti into 7 training and 3 test images per subject and fit PCA with 120 components.
4. Train an RBF SVM ($C = 10$) and a 1-NN classifier on the PCA features and compute the test accuracy.
5. Compute LBP codes, 8×8 grid histograms and the chi-square nearest-neighbour accuracy.
6. Align face pairs with YuNet, extract SFace embeddings and compare the cosine similarity with 0.363.
7. Plot eigenfaces, predictions, accuracies and verification pairs.

5. Output

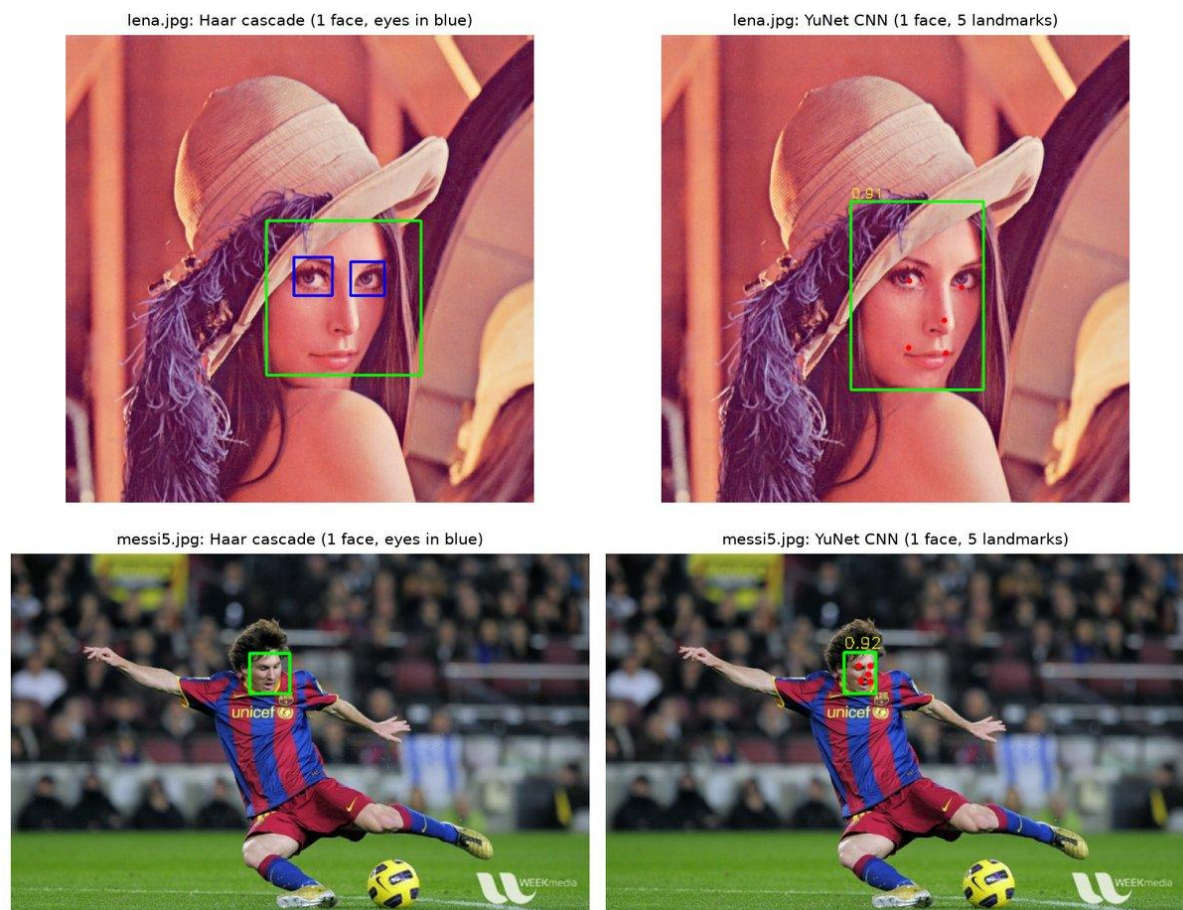


Figure 11.1: Face detection on lena and messi5 with the Haar cascade (face and eye boxes) and YuNet (box, landmarks, score).

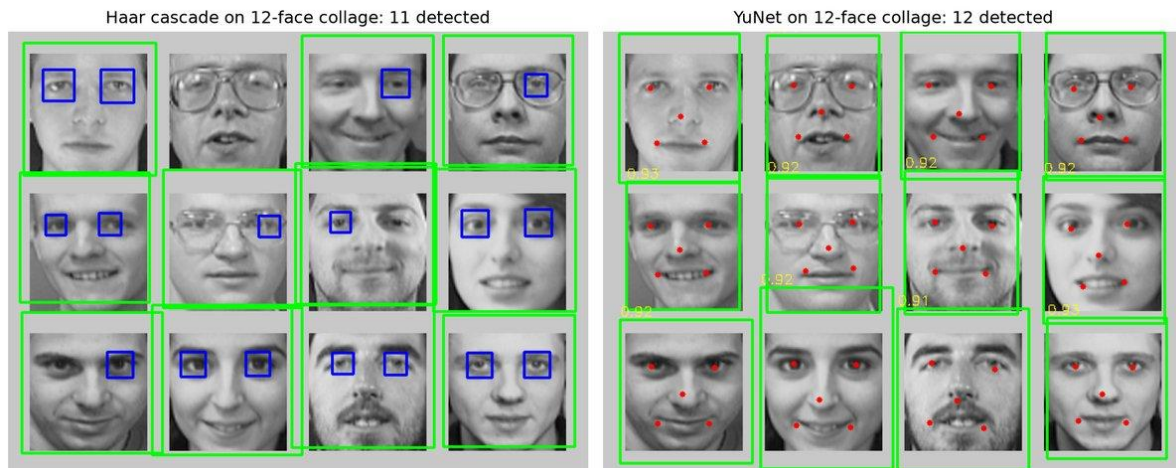


Figure 11.2: Detection on a 12-face Olivetti collage: Haar finds 11 faces, YuNet all 12.

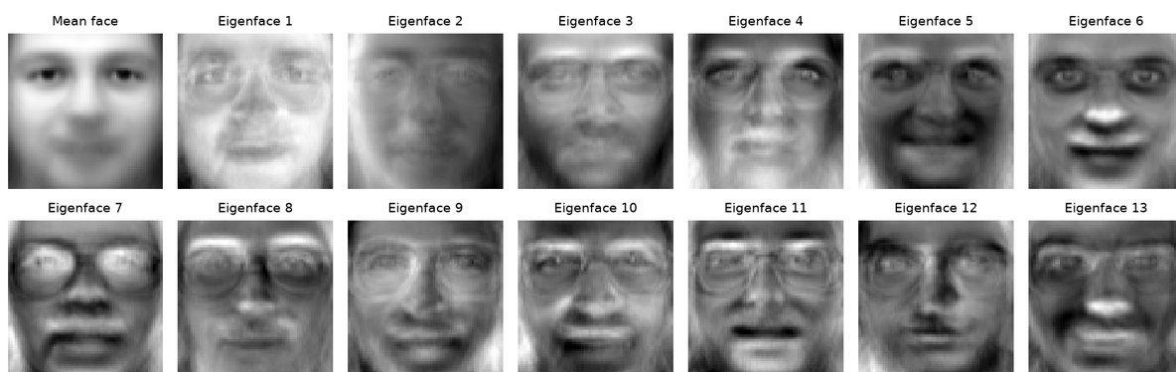


Figure 11.3: Mean face and the first 13 eigenfaces from the 280 training faces.

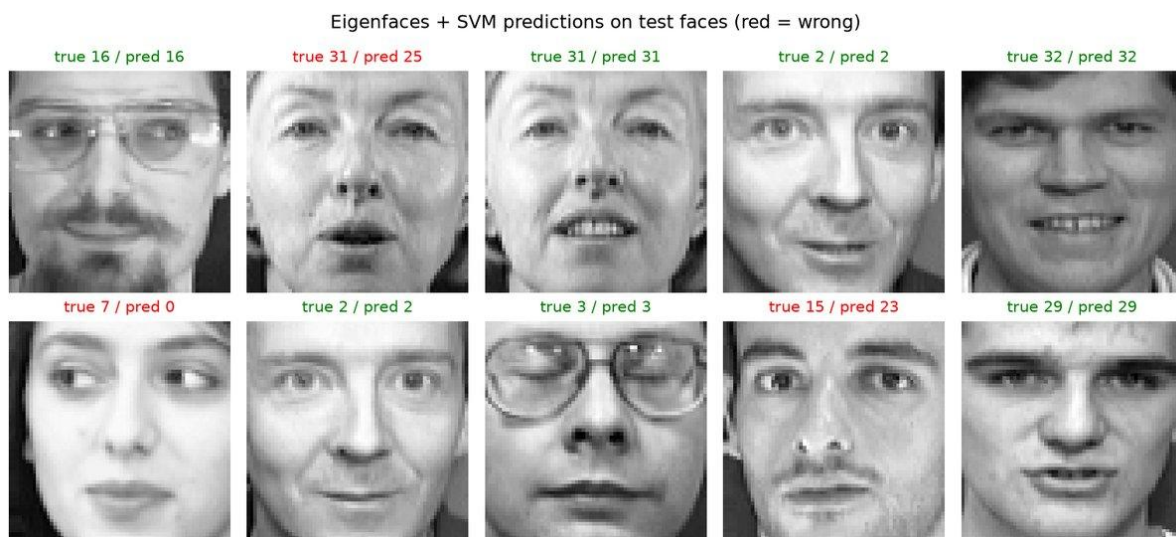


Figure 11.4: Ten test faces with the true and predicted subjects of the Eigenfaces + SVM classifier. Errors in red.

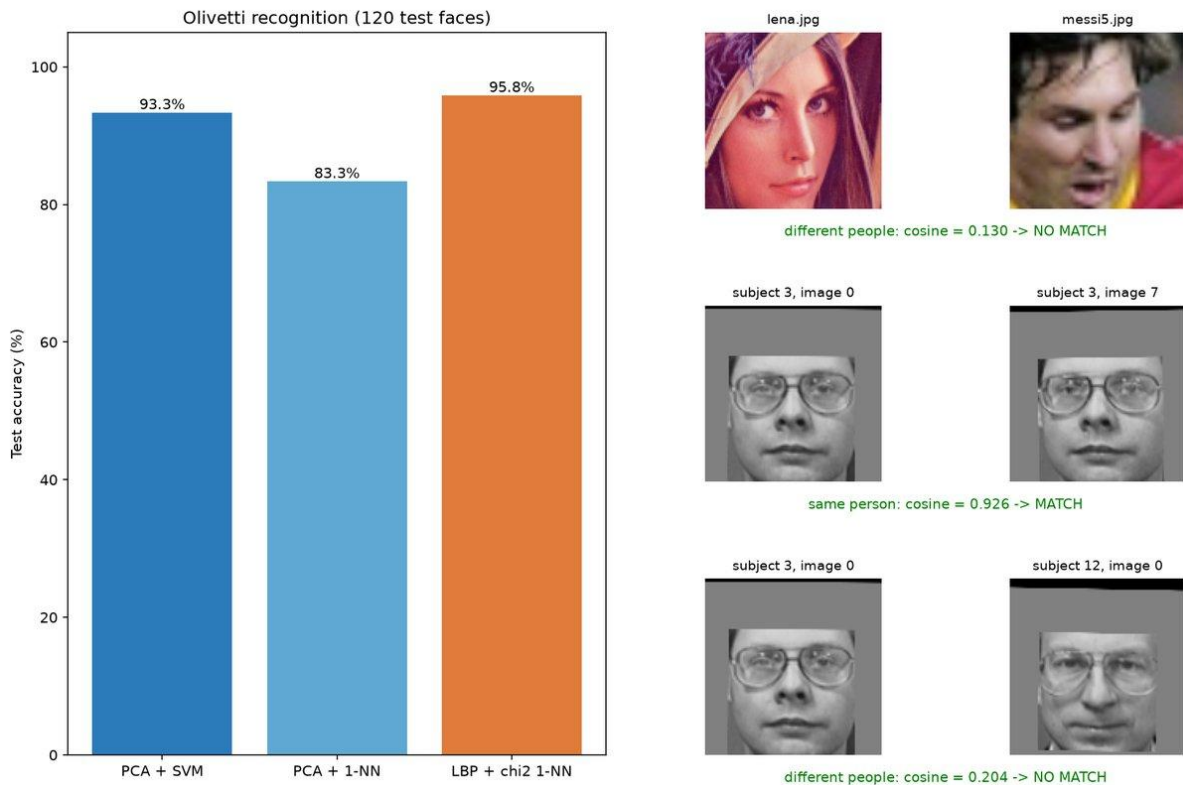


Figure 11.5: Accuracy of the three recognisers and SFace verification of three face pairs with cosine scores.

Console output

```
lena.jpg Haar faces = 1, YuNet faces = 1
messi5.jpg Haar faces = 1, YuNet faces = 1
collage Haar faces = 11 / 12, YuNet faces = 12 / 12
PCA: 120 components explain 96.2 % of the variance
Eigenfaces + SVM accuracy = 93.33 % (112 / 120)
Eigenfaces + 1-NN accuracy = 83.33 %
LBP histogram + chi-square 1-NN accuracy = 95.83 %
SFace lena.jpg vs messi5.jpg: cosine = 0.130 -> NO MATCH (threshold 0.363)
SFace subject 3, image 0 vs subject 3, image 7: cosine = 0.926 -> MATCH (threshold 0.363)
SFace subject 3, image 0 vs subject 12, image 0: cosine = 0.204 -> NO MATCH (threshold 0.363)
Confusion matrix (SVM): 33 subjects perfectly recognised, 8 misclassified test faces
Saved figures to outputs/exp11
```

6. Results and Observations

Both detectors found the face in lena.jpg and messi5.jpg (YuNet confidence 0.91 and 0.92). On the 12-face collage the Haar cascade found 11 faces; YuNet found all 12 with correct landmarks.

With 120 PCA components (96.2 percent of the variance) Eigenfaces + SVM reached 93.33 percent (112 of 120 test faces), PCA + 1-NN 83.33 percent and the LBP chi-square recogniser 95.83 percent.

SFace gave a cosine similarity of 0.926 for two images of subject 3 (match) and 0.204 and 0.130 for different people (no match). All three verification decisions were correct.

7. Conclusion

The Haar cascade is fast but missed one face of the collage; YuNet detected every face and its landmarks. On Olivetti the LBP histograms (95.8 percent) beat Eigenfaces (93.3 percent), and the SFace similarities of 0.926 for the same person against 0.204 or less for different people leave the 0.363 threshold well clear.

Experiment No. 12

Object detection from dynamic Background for Surveillance

Experiment No.	12
Title	Object detection from dynamic Background for Surveillance
Date of experiment	
Tools / software used	Python 3.11, OpenCV 5.0 (accumulateWeighted, createBackgroundSubtractorMOG2, createBackgroundSubtractorKNN, morphologyEx, findContours, pointPolygonTest), NumPy, Matplotlib
Input data	OpenCV sample videos vtest.avi (768 x 576, fixed camera, pedestrians; first 300 frames) and tree.avi (320 x 240, 68 frames of a tree swaying in the wind seen through a window, with a hand entering the view at about frame 53).
Program file	exp12_surveillance_bg_subtraction.py

1. Aim

To detect moving objects in surveillance video by modelling and subtracting the background, to compare frame differencing with the adaptive MOG2 and KNN models on static and dynamic backgrounds, and to build a zone-intrusion alarm on the detections.

2. Objectives

- Implement running-average differencing and use the MOG2 and KNN background subtractors.
- Clean the masks and extract bounding boxes of moving objects, including on a dynamic background.
- Implement a restricted-zone intrusion alarm.

3. Theory

Background subtraction compares each frame from a fixed camera with a background model and labels every pixel as background or foreground. The model must follow illumination changes and represent backgrounds that move, such as leaves. The simplest model is a running average with learning rate α ; a pixel is foreground when it differs from the model by more than τ . With one value per pixel it cannot represent a multi-modal background, and a moving object leaves a ghost trail.

$$B_t = (1 - \alpha)B_{t-1} + \alpha I_t, \quad F_t(x, y) = \begin{cases} 1 & |I_t(x, y) - B_t(x, y)| > \tau \\ 0 & \text{otherwise} \end{cases}$$

MOG2 models each pixel with a mixture of K Gaussians. Components with high weight and low variance are background, and a value that matches none of them is foreground. Shadows are flagged with the value 127 so a threshold removes them. KNN keeps recent samples per pixel and calls a value background when at least k samples lie within a distance threshold. Median filtering and morphology clean the masks, contours give the boxes, and a box with its foot point inside the polygon raises the alarm.

$$p(I_t) = \sum_{k=1}^K w_k \mathcal{N}(I_t; \mu_k, \sigma_k^2)$$

4. Algorithm / Procedure

1. Feed every frame to the running-average differencer ($\alpha = 0.02$, $\tau = 30$), MOG2 and KNN (history 300, shadows on).
2. Threshold each raw mask at 255 to drop the shadow label, then apply median, opening, closing and dilation.
3. Find external contours, discard components below 400 px and compute the bounding boxes.
4. Record the foreground fraction, object count and processing time per frame and store background images at frames 30 and 300.
5. Repeat on tree.avi to compare the false foreground from moving leaves with the response to the hand.
6. Test the foot point of every MOG2 box against a restricted zone polygon with pointPolygonTest and raise alarms.
7. Draw masks, boxes, background images, plots and alarm frames.

5. Output



Figure 12.1: Frame 120 of vtest.avi, raw and cleaned MOG2 masks, differencing and KNN masks, and the detected objects.

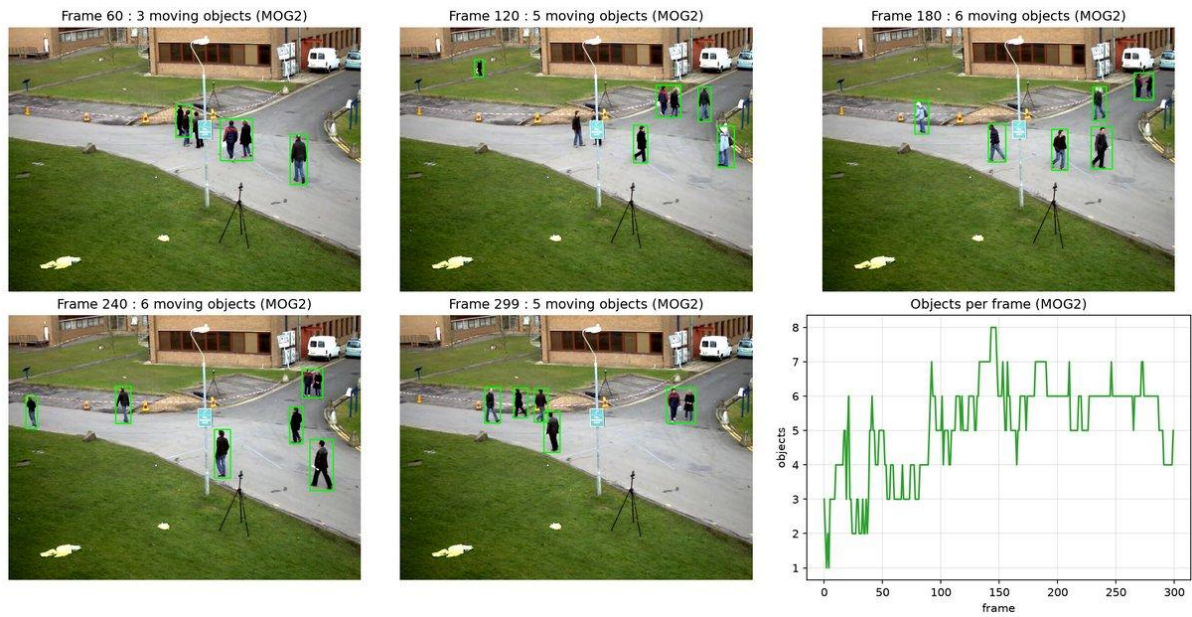


Figure 12.2: MOG2 bounding boxes at frames 60 to 299 and the number of objects per frame.

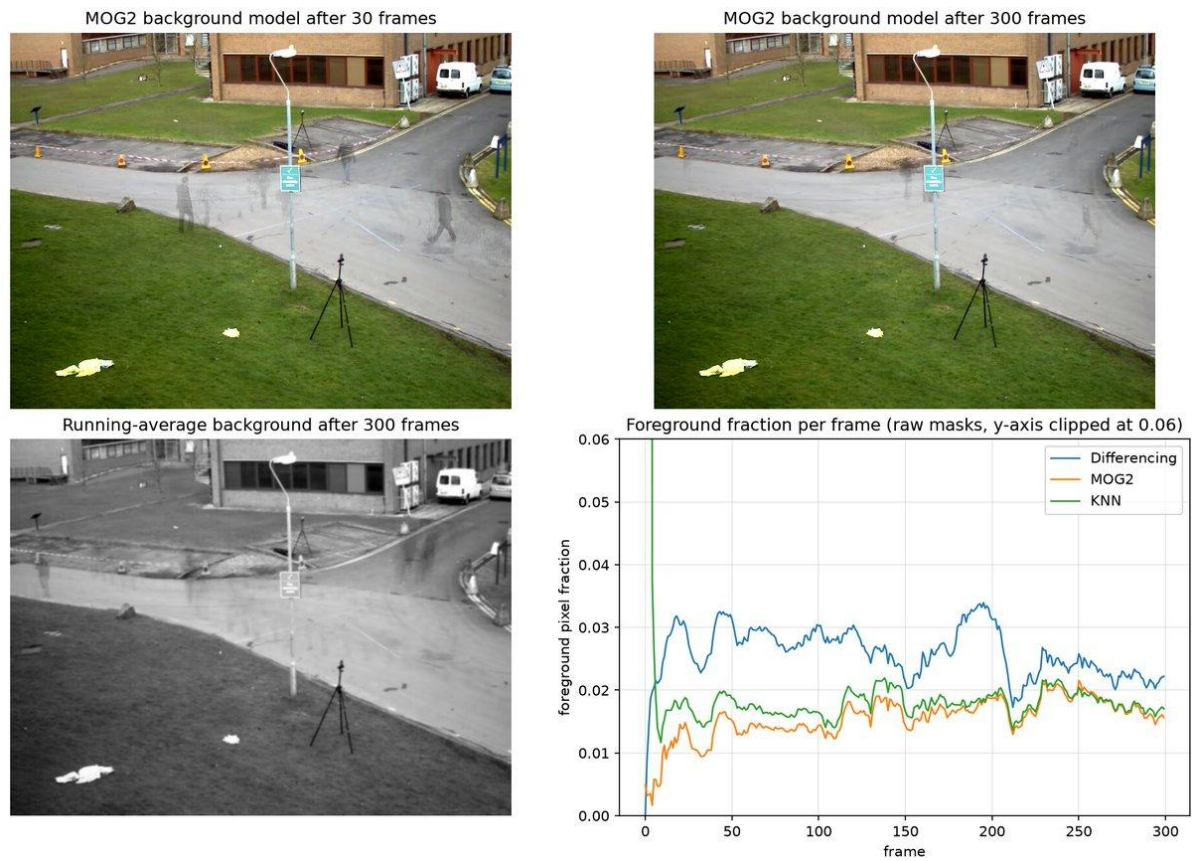


Figure 12.3: MOG2 background after 30 and 300 frames, the running-average background, and the foreground fraction over time.

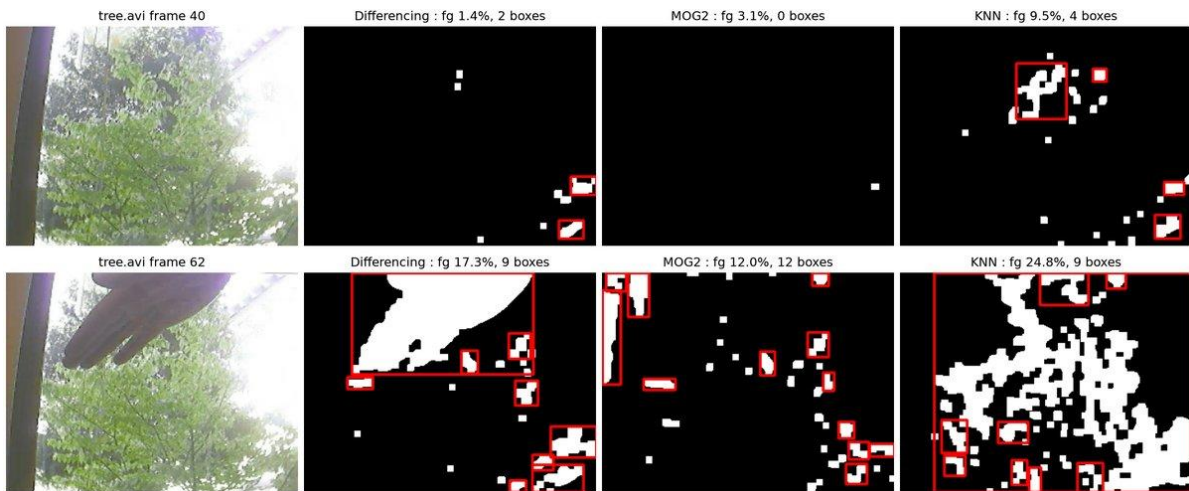


Figure 12.4: Dynamic background test on tree.avi at frame 40 (no object) and frame 62 (hand) for the three methods.

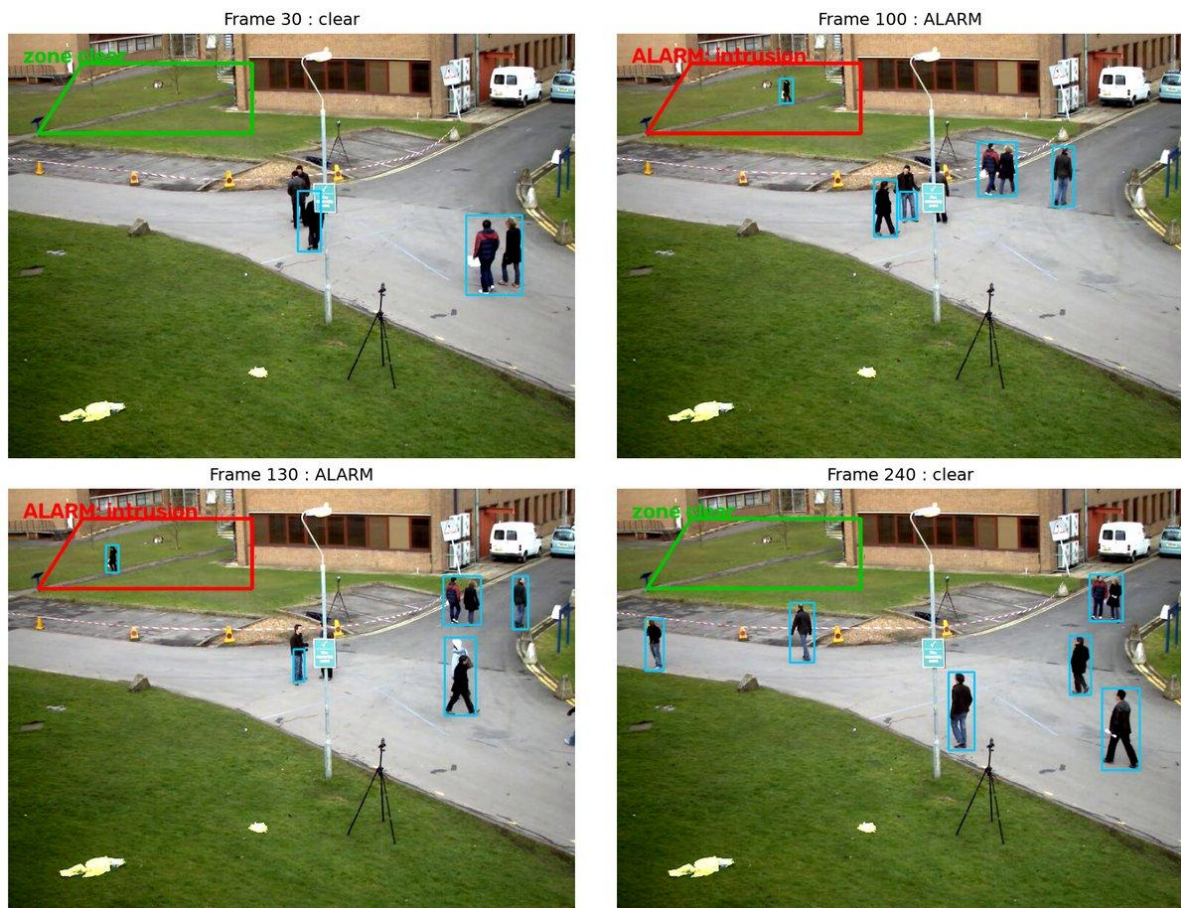


Figure 12.5: Zone-intrusion alarm: the lawn polygon is green when clear and red with an ALARM label when entered.

Console output

```
vtest.avi (768x576), 300 frames, min blob area 400 px
Differencing : mean fg fraction = 0.0261    mean objects/frame = 5.90    162.6 fps (apply + clean-up)
MOG2          : mean fg fraction = 0.0160    mean objects/frame = 5.25    133.8 fps (apply + clean-up)
KNN           : mean fg fraction = 0.0178    mean objects/frame = 5.47    122.2 fps (apply + clean-up)
tree.avi (320x240), 68 frames: swaying leaves, a hand enters at about frame 53
Differencing : mean fg fraction frames 20-52 (no object) = 0.0102    frames 55-67 (hand) = 0.1694
false boxes/frame (20-52) = 1.45
```

```

MOG2      : mean fg fraction frames 20-52 (no object) = 0.0263   frames 55-67 (hand) = 0.1305
false boxes/frame (20-52) = 0.24
KNN       : mean fg fraction frames 20-52 (no object) = 0.0829   frames 55-67 (hand) = 0.2482
false boxes/frame (20-52) = 3.45
Zone intrusion (MOG2 boxes, foot point inside polygon): 76 of 300 frames raised an alarm, first alarm
at frame 82
Outputs written to E:\Plant-Disease-Detection-and-Analysis-System\LAB\outputs\exp12

```

6. Results and Observations

On 300 vtest.avi frames the differencer, MOG2 and KNN marked 2.61, 1.60 and 1.78 percent of the pixels as foreground and detected 5.90, 5.25 and 5.47 objects per frame at 162.6, 133.8 and 122.2 fps. MOG2 gave the tightest masks after shadow removal.

On tree.avi with no object present the false foreground was 1.02, 2.63 and 8.29 percent, with 1.45, 0.24 and 3.45 false boxes per frame. MOG2 separated the swaying leaves from the hand best.

The intrusion detector raised an alarm in 76 of 300 frames, the first at frame 82 when a person walks onto the lawn. Frames with nobody in the zone stayed clear.

7. Conclusion

MOG2 gave the fewest false boxes on the swaying tree and removed shadows, differencing left ghost trails, and KNN had not learned the background within the 68 available frames. The cleaned MOG2 mask gives the object boxes for the zone alarm directly.

Experiment No. 13

Content based video retrieval

Experiment No.	13
Title	Content based video retrieval
Date of experiment	
Tools / software used	Python 3.11, OpenCV 5.0 (cvtColor, Sobel, imencode/imdecode), PyTorch 2.10 + torchvision 0.25 (ResNet-18 ImageNet weights on GPU), NumPy, Matplotlib
Input data	Four OpenCV sample videos: Megamind.avi and Megamind_buggy.avi (270 frames each, animation with cuts; the second file is a corrupted copy of the first with a green bar artefact), vtest.avi (795 frames, fixed surveillance camera) and tree.avi (68 frames, swaying tree and a hand). All frames are resized to 320 x 240.
Program file	exp13_video_retrieval.py

1. Aim

To build a content-based video retrieval system that cuts videos into shots, describes each shot by visual features taken from its frames, and returns the shots most similar to a query clip.

2. Objectives

- Detect shot boundaries from colour-histogram differences and pick a key frame for each shot.
- Describe each shot with colour, edge and deep CNN features and rank the database against transformed queries.
- Evaluate retrieval with the rank of the true shot, mean reciprocal rank (MRR) and precision at 1.

3. Theory

Content based video retrieval (CBVR) finds video segments by what they look like. The videos are cut into shots, each shot is summarised by a feature vector, and a query is answered by ranking the stored vectors by distance. A hard cut changes the colour histogram abruptly, so a cut is declared where the distance between the normalised histograms $h(t)$ of consecutive frames exceeds a threshold. The middle frame of each clip is its key frame.

$$d(t) = \frac{1}{2} \sum_b |h_b(t) - h_b(t-1)|$$

Three descriptors are used: an HSV colour histogram ($8 \times 8 \times 4$ bins), an edge-orientation histogram (16 bins weighted by gradient magnitude) and the L_2 -normalised 512-dimensional ResNet-18 embedding of the key frame. Histograms are compared with the chi-square distance and deep features with the cosine distance. Each distance is divided by its database mean before the three are fused as $d = 0.4\bar{d}_{\text{colour}} + 0.2\bar{d}_{\text{edge}} + 0.4\bar{d}_{\text{deep}}$. Retrieval quality is measured by $\text{MRR} = \frac{1}{Q} \sum_q 1/\text{rank}_q$ and by precision at 1.

$$d_{\chi^2}(p, q) = \frac{1}{2} \sum_b \frac{(p_b - q_b)^2}{p_b + q_b}, \quad d_{\cos}(\mathbf{p}, \mathbf{q}) = 1 - \mathbf{p} \cdot \mathbf{q}$$

4. Algorithm / Procedure

1. Read every video at 320×240 and compute the normalised HSV histogram of each frame.
2. Declare a cut where the histogram distance exceeds 0.12 (minimum shot length 8 frames) and split shots longer than 60 frames.
3. For every clip store the key frame, the mean HSV histogram, the edge-orientation histogram and the ResNet-18 embedding.
4. Build five queries: crop, brightness change, horizontal flip, down-scaling with JPEG compression, and later frames of a shot.
5. Compute chi-square and cosine distances from each query to every clip, scale by the mean and combine with weights 0.4, 0.2, 0.4.
6. Rank the database, record the rank of the true clip per feature type and compute MRR, P@1 and mean rank.
7. Plot the shot-boundary curve, key-frame gallery, top-5 results and metric chart.

5. Output

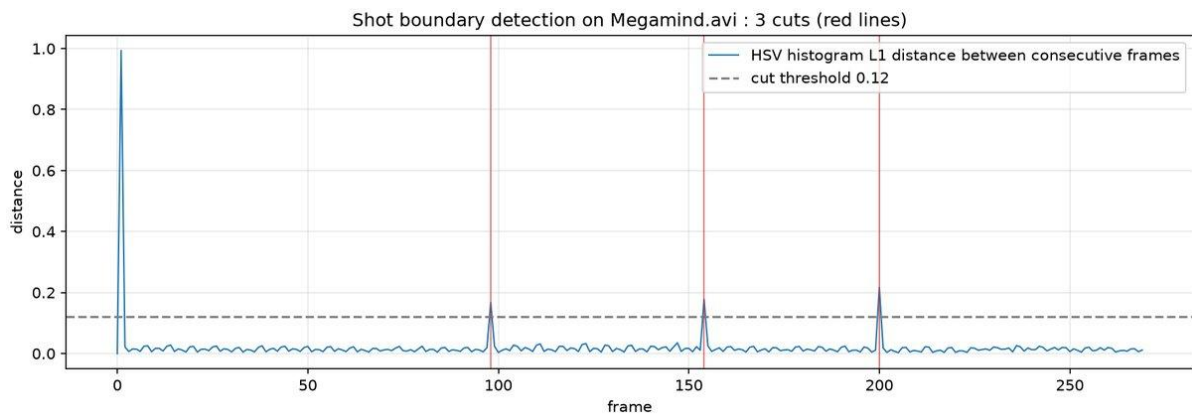


Figure 13.1: Histogram distance between consecutive frames of Megamind.avi. The three cuts at frames 98, 154 and 200 exceed the 0.12 threshold.

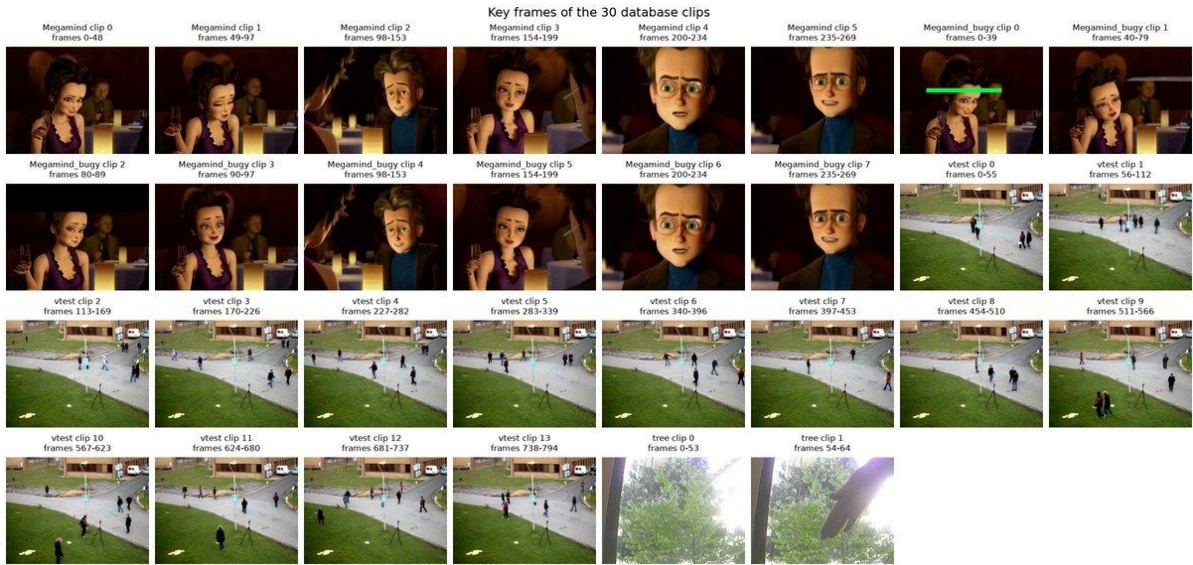


Figure 13.2: Key frames of the 30 database clips with their frame ranges.

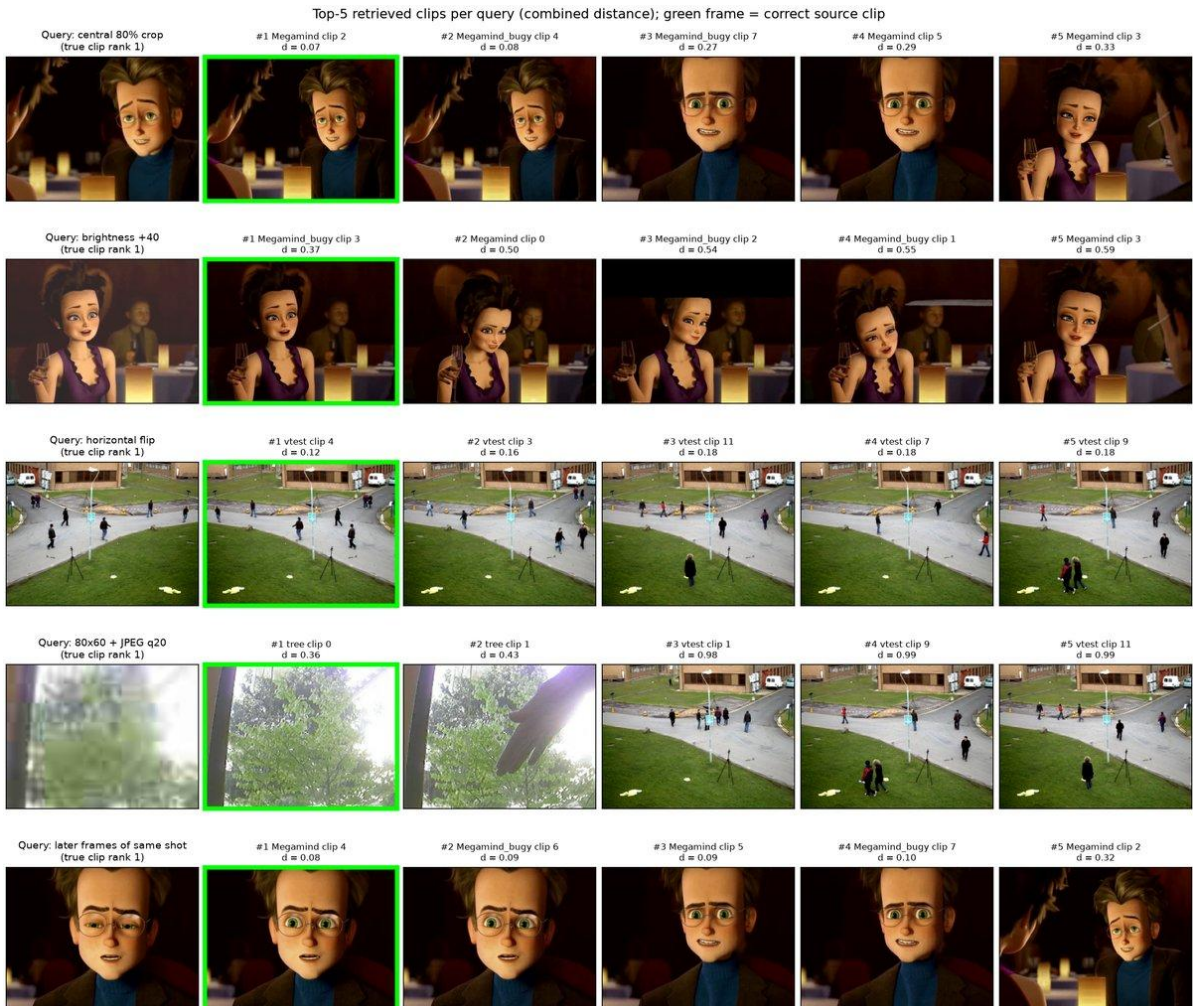


Figure 13.3: Five query clips (left) and their top-5 results by combined distance. The green frame marks the true source clip.

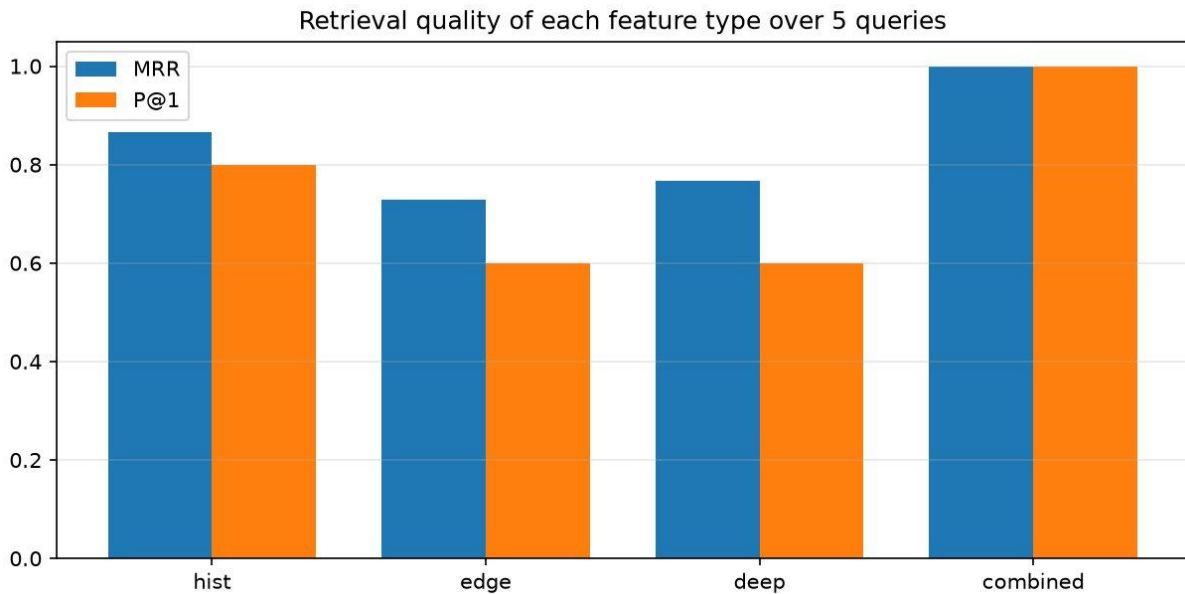


Figure 13.4: MRR and precision at 1 for colour, edge, deep and combined features over the five queries.

Console output

```
Megamind.avi      : 270 frames, 4 shots detected, 6 database clips after splitting long shots
Megamind_buggy.avi : 270 frames, 7 shots detected, 8 database clips after splitting long shots
vtest.avi         : 795 frames, 1 shots detected, 14 database clips after splitting long shots
tree.avi          : 68 frames, 2 shots detected, 2 database clips after splitting long shots
Database size = 30 clips

Retrieval results (rank of the true clip out of 30, combined distance = 0.4 hist + 0.2 edge + 0.4
deep, each scaled by its mean)
  central 80% crop      (true = Megamind.avi clip 2) : hist rank 1 edge rank 2 deep rank 2
combined rank 1
  brightness +40        (true = Megamind_buggy.avi clip 3) : hist rank 3 edge rank 1 deep rank
1 combined rank 1
  horizontal flip        (true = vtest.avi clip 4) : hist rank 1 edge rank 7 deep rank 1
combined rank 1
  80x60 + JPEG q20      (true = tree.avi clip 0) : hist rank 1 edge rank 1 deep rank 1
combined rank 1
  later frames of same shot (true = Megamind.avi clip 4) : hist rank 1 edge rank 1 deep rank 3
combined rank 1

feature    MRR    P@1    mean rank
hist       0.867  0.80   1.40
edge       0.729  0.60   2.40
deep       0.767  0.60   1.60
combined   1.000  1.00   1.00
... (output truncated) ...
```

6. Results and Observations

Shot detection found 4 shots in Megamind.avi (cuts at frames 98, 154 and 200), 7 in Megamind_buggy.avi, 1 in vtest.avi and 2 in tree.avi. After splitting the long shots the database held 30 clips.

Each single feature failed somewhere. The colour histogram ranked the true clip third under the brightness change, the edge histogram ranked it seventh under the flip, and the deep feature preferred near-duplicate clips (ranks 2 and 3). The combined distance ranked the true clip first for all five queries.

MRR was 0.867 (colour), 0.729 (edge), 0.767 (deep) and 1.000 (combined). P@1 was 0.80, 0.60, 0.60 and 1.00, and the mean ranks were 1.40, 2.40, 1.60 and 1.00.

7. Conclusion

Histogram differences cut the videos into shots, and colour, edge and deep features each rank the database for a query. No single feature survived every transformation, but their weighted fusion put the correct clip at rank 1 for all five queries.

Experiment No. 14

Construct 3D model from single image

Experiment No.	14
Title	Construct 3D model from single image
Date of experiment	
Tools / software used	Python 3.11, PyTorch 2.10 with CUDA (MiDaS v2.1 small via torch.hub, timm), OpenCV 5.0, NumPy, Matplotlib (mplot3d, Poly3DCollection)
Input data	OpenCV sample photographs data/building.jpg (868 x 600) and data/home.jpg (512 x 384), one PlantVillage potato leaf image (Potato__Late_blight, 256 x 256), the MiDaS weights data/models/midas_v21_small_256.pt, and a synthetic 128 x 128 rendering of a Lambertian sphere of radius 45 px.
Program file	exp14_3d_from_single_image.py

1. Aim

To build a 3D model of a scene from a single photograph by estimating a dense depth map with the MiDaS monocular depth network and back-projecting it through a pinhole camera model, and to show the classical shape-from-shading approach on a synthetic Lambertian sphere.

2. Objectives

- To understand why depth from a single image is ill-posed and which cues make it solvable.
- To run the MiDaS monocular depth network and convert its output into a textured mesh and a PLY point cloud.
- To implement classical shape from shading for a Lambertian sphere.

3. Theory

A pinhole camera maps every point of a viewing ray to the same pixel, so depth is lost. A 3D model from one image needs extra assumptions (a reflectance model, texture, perspective cues or learned priors) and is known only up to scale.

For a Lambertian surface with albedo ρ lit from direction $\mathbf{l}$, the irradiance is $E = \rho \mathbf{n} \cdot \mathbf{l}$ with normal $\mathbf{n} = (-p, -q, 1)/\sqrt{1 + p^2 + q^2}$, $p = \partial Z/\partial x$, $q = \partial Z/\partial y$. With the light along the viewing direction this becomes the eikonal equation, solved by fast sweeping.

$$E = \frac{\rho}{\sqrt{1 + p^2 + q^2}} \Rightarrow |\nabla Z| = \sqrt{\frac{\rho}{E^2} - 1}$$

MiDaS is a convolutional network trained with a scale- and shift-invariant loss; its output is a relative inverse depth map (bright means near). Normalising it to $n \in [0,1]$, mapping to a relative depth Z and back-projecting with focal length $f = 1.2W$ gives a point cloud, and the pixel grid turns it into a textured mesh.

$$X = \frac{(u - c_x)Z}{f}, \quad Y = \frac{(v - c_y)Z}{f}, \quad Z = \frac{1}{0.2 + 0.8n}$$

4. Algorithm / Procedure

1. Load MiDaS v2.1 small with the local weights on the GPU, together with its input transform.
2. Run the network on each image, time it, and resample the inverse depth to the original image size.
3. Normalise to $n \in [0,1]$, set $Z = 1/(0.2 + 0.8n)$, back-project with $f = 1.2W$ and write a coloured PLY.
4. Sub-sample the grid, form quadrilaterals, drop sky cells and cells with depth jumps above 10 %, and render the textured mesh.
5. Render a Lambertian sphere lit from the camera, compute the slowness $\sqrt{1/E^2 - 1}$ and solve the eikonal equation by fast sweeping.
6. Recover the height $Z = r - T$, compare it with the true sphere (RMSE) and plot the maps and the 3D surface.

5. Output

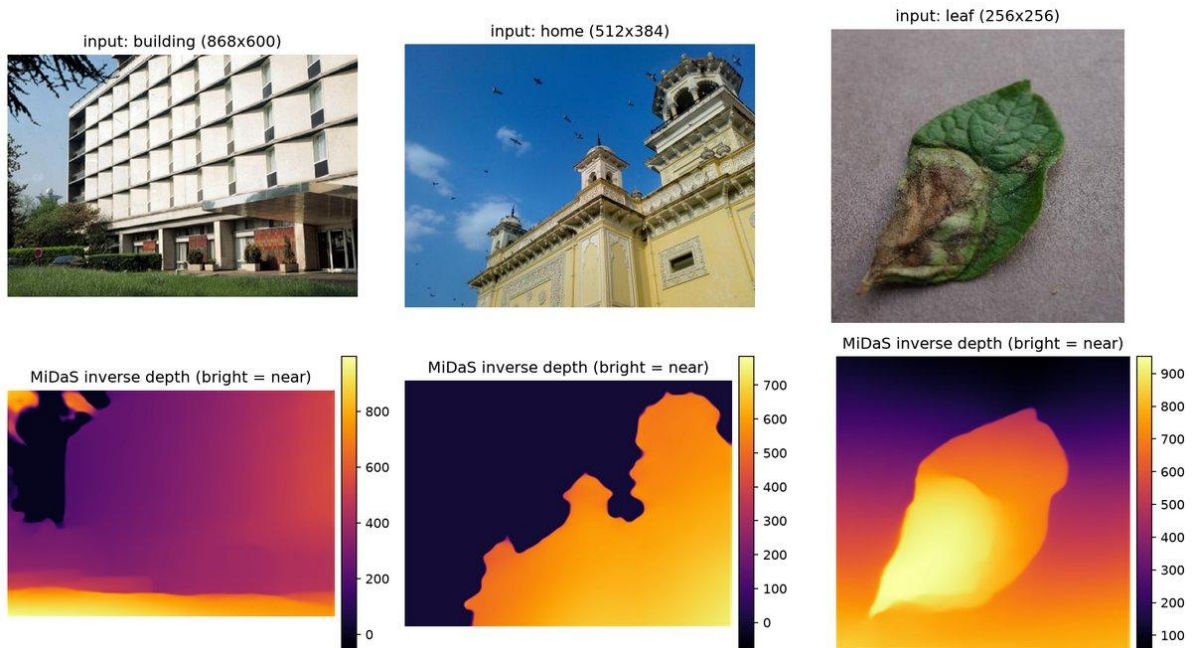


Figure 14.1: Input photographs (top) and MiDaS inverse depth maps (bottom, bright means near).

3D model from a single photograph: MiDaS depth back-projected with an assumed pinhole camera

building: textured mesh (elev 22, azim -55)

building: textured mesh (elev 10, azim -115)

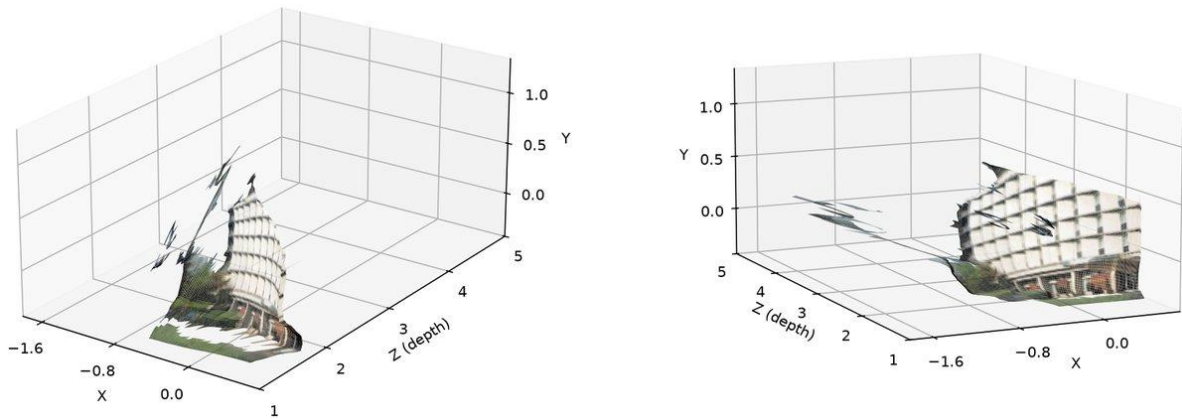


Figure 14.2: Textured 3D mesh of the building from the MiDaS depth and an assumed pinhole camera (18 018 quadrilaterals).

home: point cloud (30 000 of 196608 points)

leaf: point cloud (30 000 of 65536 points)

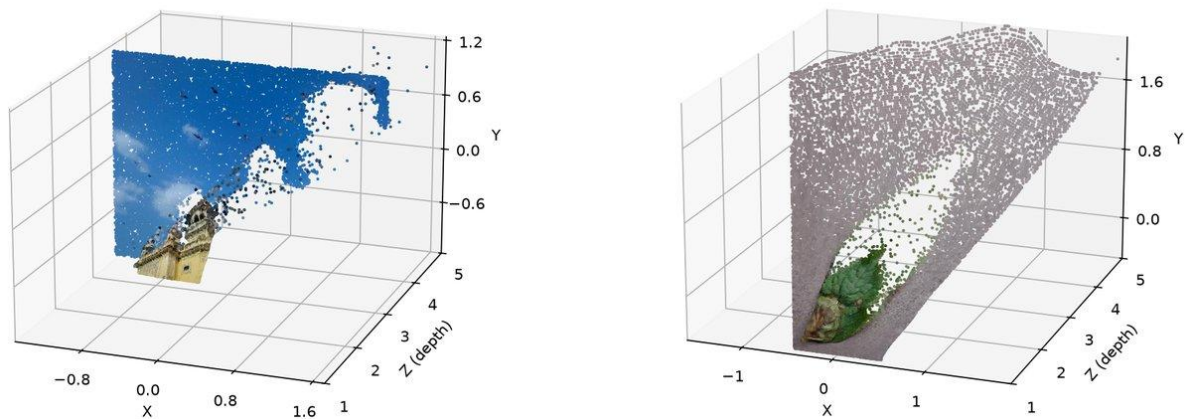


Figure 14.3: Coloured point clouds of the home and leaf images (30 000 points shown for each).

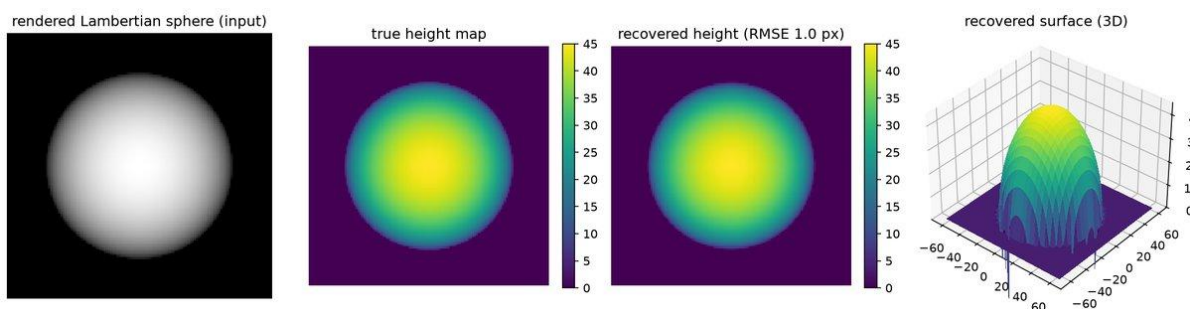


Figure 14.4: Shape from shading on a Lambertian sphere: input, true height, recovered height (RMSE 1.05 px) and the 3D surface.

Console output

```

MiDaS v2.1 small loaded on cuda; parameters: 21.3 M
building : 868x600 px, inference 30.7 ms, inverse depth -56.9..996.8, relative depth 1.0..5.0, PLY
points 130200
home      : 512x384 px, inference 36.1 ms, inverse depth -81.9..784.3, relative depth 1.0..5.0, PLY
points 49152
leaf      : 256x256 px, inference 40.0 ms, inverse depth 54.2..952.1, relative depth 1.0..5.0, PLY
points 16384
Shape from shading (eikonal fast sweeping, 1.5 s): sphere radius 45 px, height RMSE 1.05 px (2.3 % of
the radius) over 6312 pixels with E > 0.1
Textured mesh of 'building': 18018 quads kept after removing depth discontinuities
Figures written to E:\Plant-Disease-Detection-and-Analysis-System\LAB\outputs\exp14

```

6. Results and Observations

MiDaS v2.1 small (21.3 million parameters) predicted inverse depth in 31 to 40 ms per image on the RTX 3050 GPU. The values are relative, with unknown scale and shift.

Back-projection produced point clouds of 130 200, 49 152 and 16 384 vertices, and a textured mesh of the building with 18 018 quadrilaterals and no streaks between near and far surfaces.

Shape from shading recovered the sphere height with an RMSE of 1.05 px (2.3 % of the 45 px radius) over 6 312 pixels in 1.5 s. The errors grow only near the silhouette.

7. Conclusion

Shape from shading recovers the shape when the reflectance and the lighting are known, as on the sphere, and the MiDaS depth map produced textured meshes and point clouds for the three test photographs. In both cases the absolute scale and the occluded back sides stay unknown.